



Building Agents on AWS

Isaac Ibrahim

GTM Specialist SA – AI/ML

isaacibr@amazon.com



What is Agentic AI?

(Partially/fully) **Autonomous** software systems that leverage AI to **reason, plan, and complete** tasks on behalf of humans or systems

Workflows



Fully autonomous

Autonomy level

Agents need the right model for the job



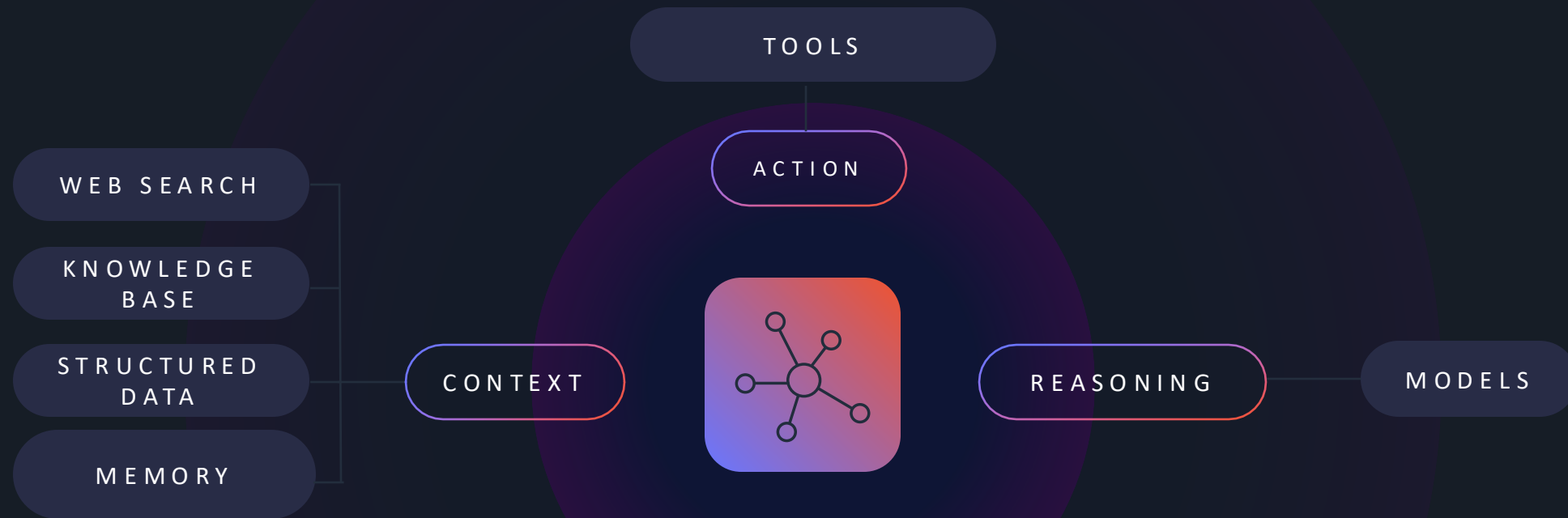
REASONING

MODELS

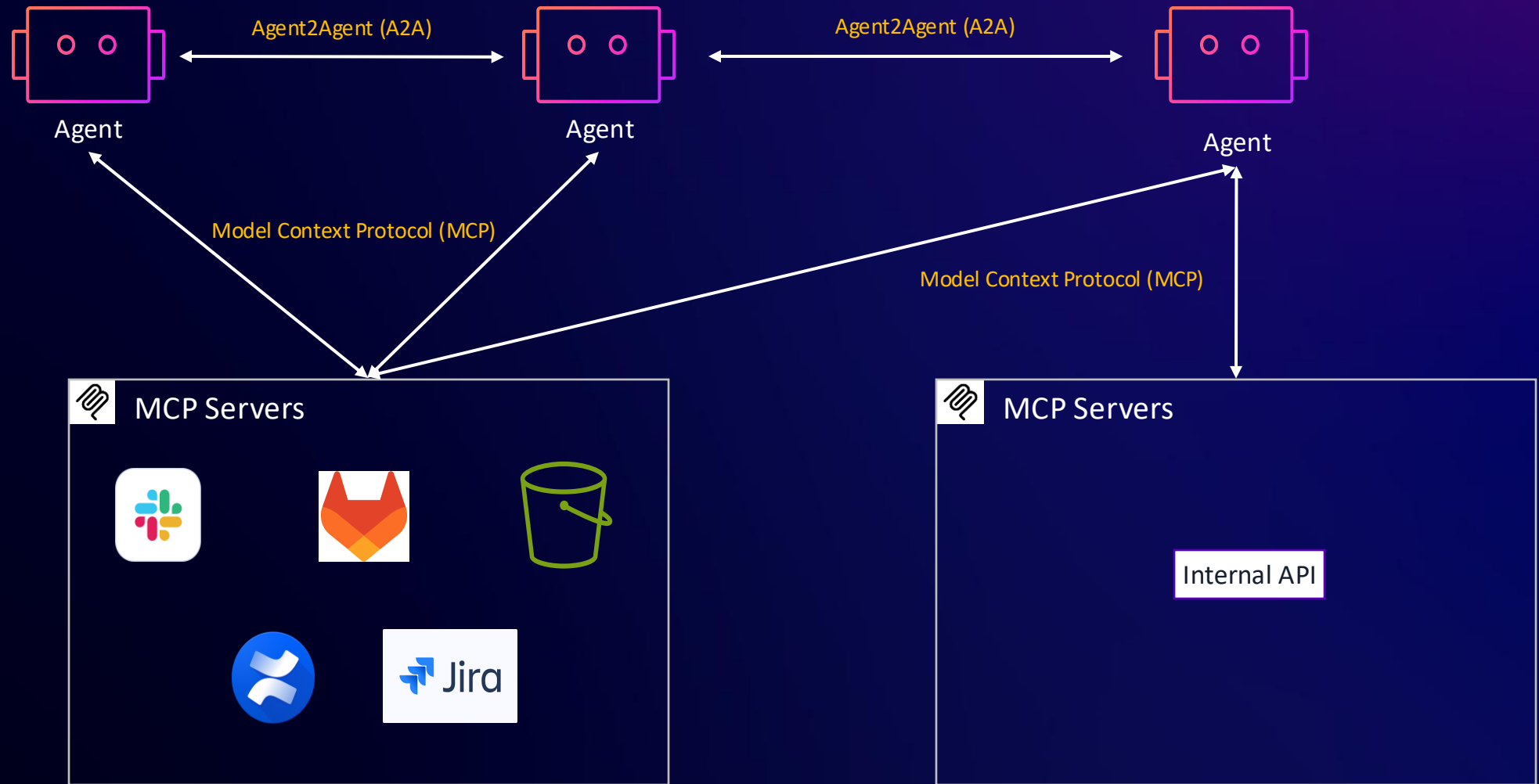
Agents require context to be useful



Agents use tools to take action



What does MCP and A2A solve?



How to standardize agent-to-tool communication?

How do agents built on different frameworks by different teams communicate with each other?

Model Context Protocol (MCP)

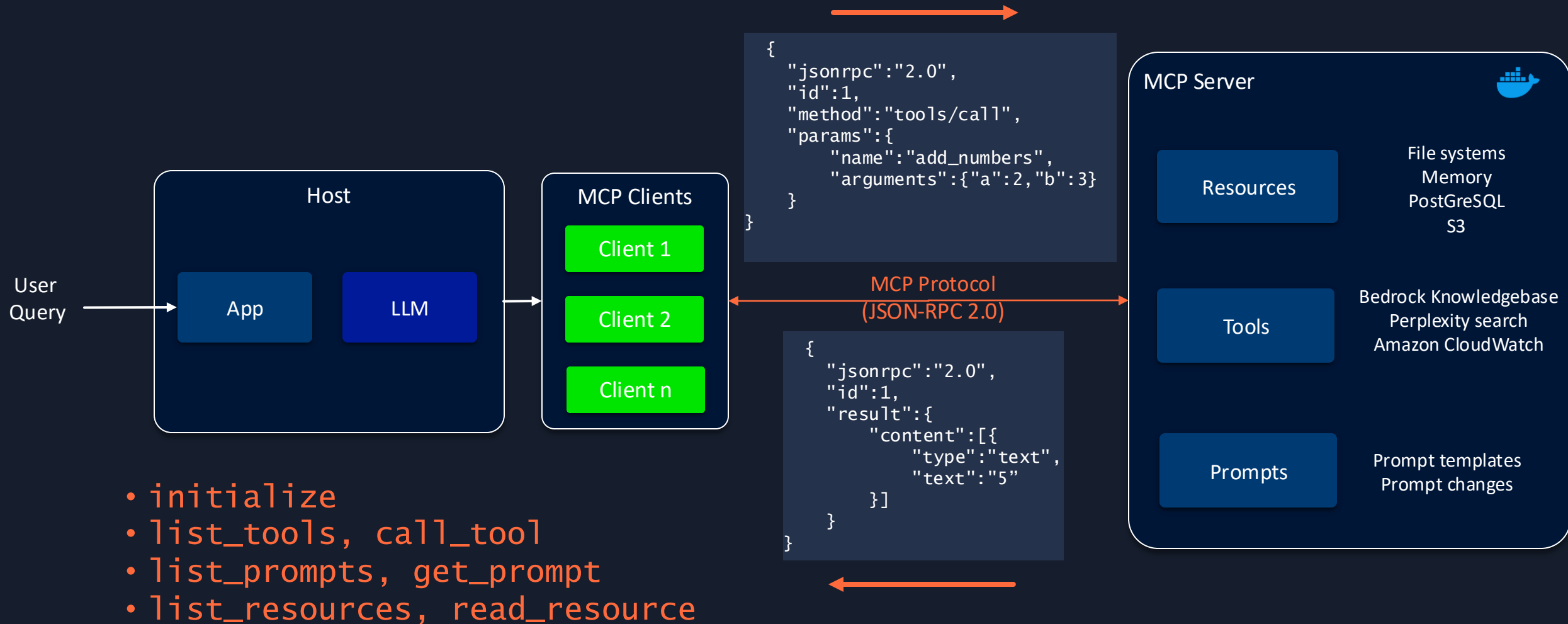
Open Protocol for integrating **tools**, **prompts**, **resources**, and **agents** for Agentic Applications

Share contextual information with LLMs

Expose tools and capabilities to AI systems

Build composable integrations and workflows

MCP Components



Multi-agent systems

Four patterns, different tradeoffs



Orchestrator / Router

Hub dispatches to specialists. Hub-and-spoke.



Swarm

Peer-to-peer; agents hand off laterally.



Hierarchical

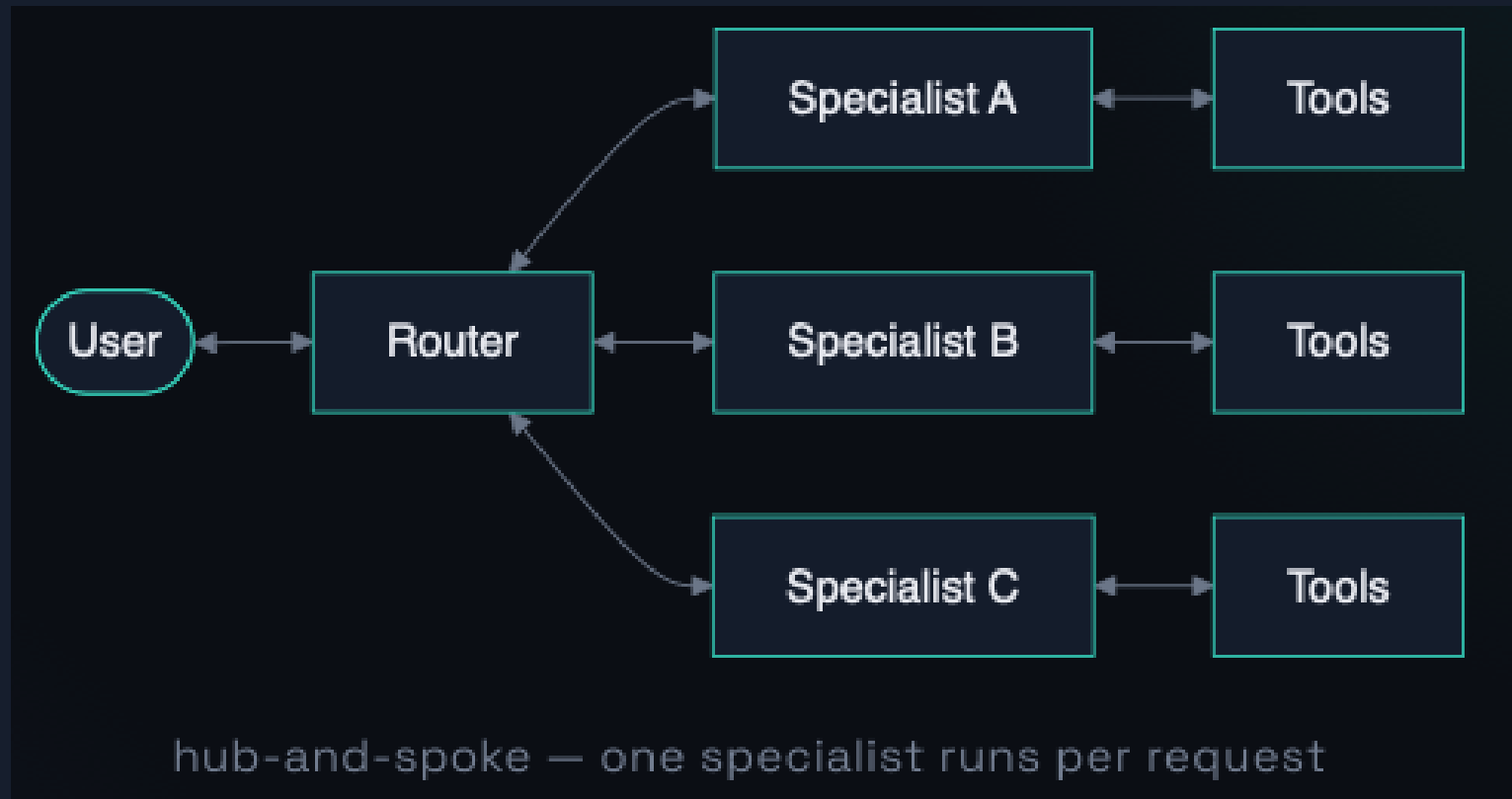
Executive → managers → workers tree.



Supervisor + Parallel

Fan out concurrently, then merge.

Pattern A: Orchestrator / Router



Use-cases

Customer support & service desks

- Telco/utility store assistant
- Bank/fintech support
- E-commerce assistant
- HR assistant

Regulated / knowledge-heavy domains

- Healthcare front door
- Insurance serving
- Legal/compliance intake
- Marketing ops

UI-menu-driven internal tools

Predefined action buttons ("Check Order," "Cancel Request," "Subscriber Lookup")



When it fits

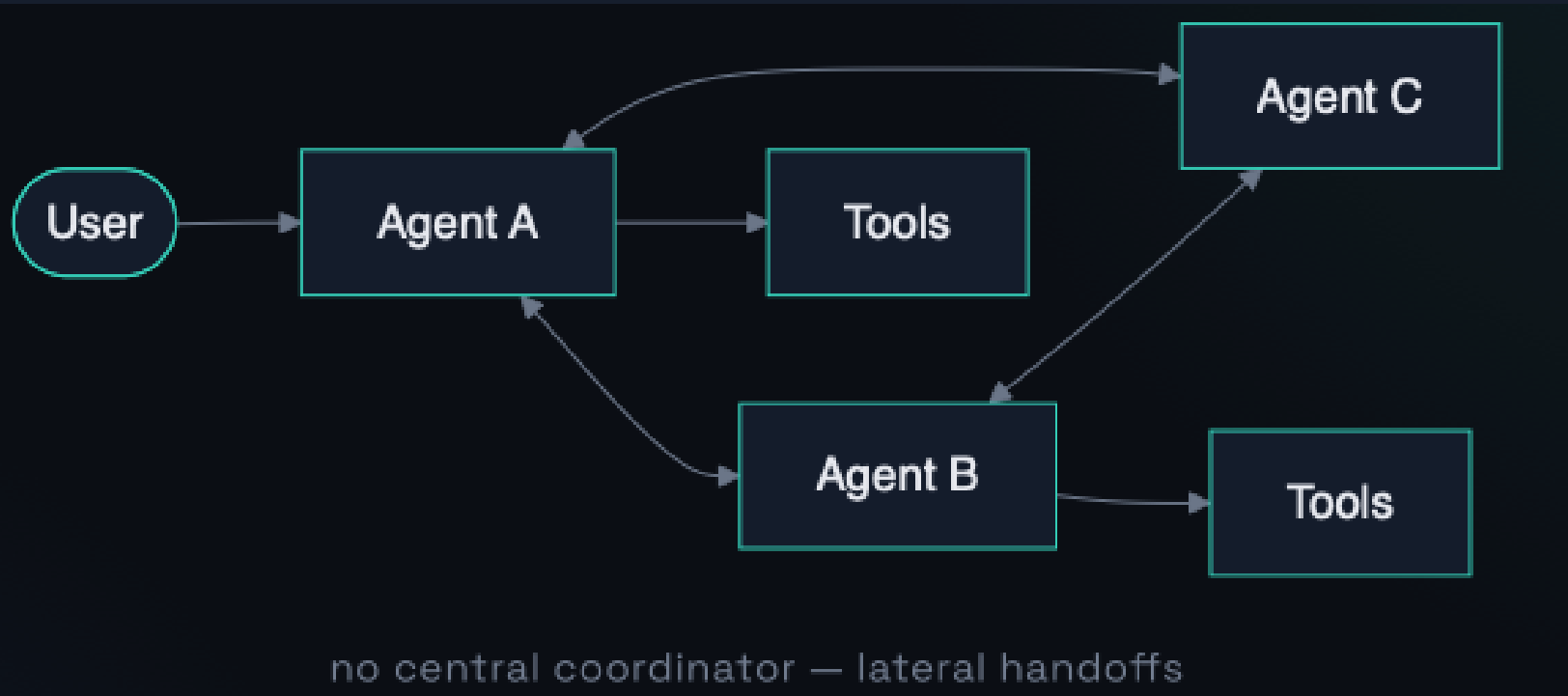
- Requests map to a **single, well-bounded domain**.
- Intent is **classifiable**.
- **Specialists are independent**.



Watch out

- **Single point of failure**.
- Cross-domain tasks need **multiple sequential dispatches**.

Pattern B: Swarm (peer-to-peer)



Use-cases

Exploration / ideation

- Brainstorming / research synthesis
- Solution-space / design exploration
- Red-team / blue-team adversarial setups
- Argument stress-testing

Quality via critique

- Draft → critique → revise loops
- Multi-perspective evaluation

Open-ended, path-unknown problems

- Complex reasoning/planning where
- Group simulation



When it fits

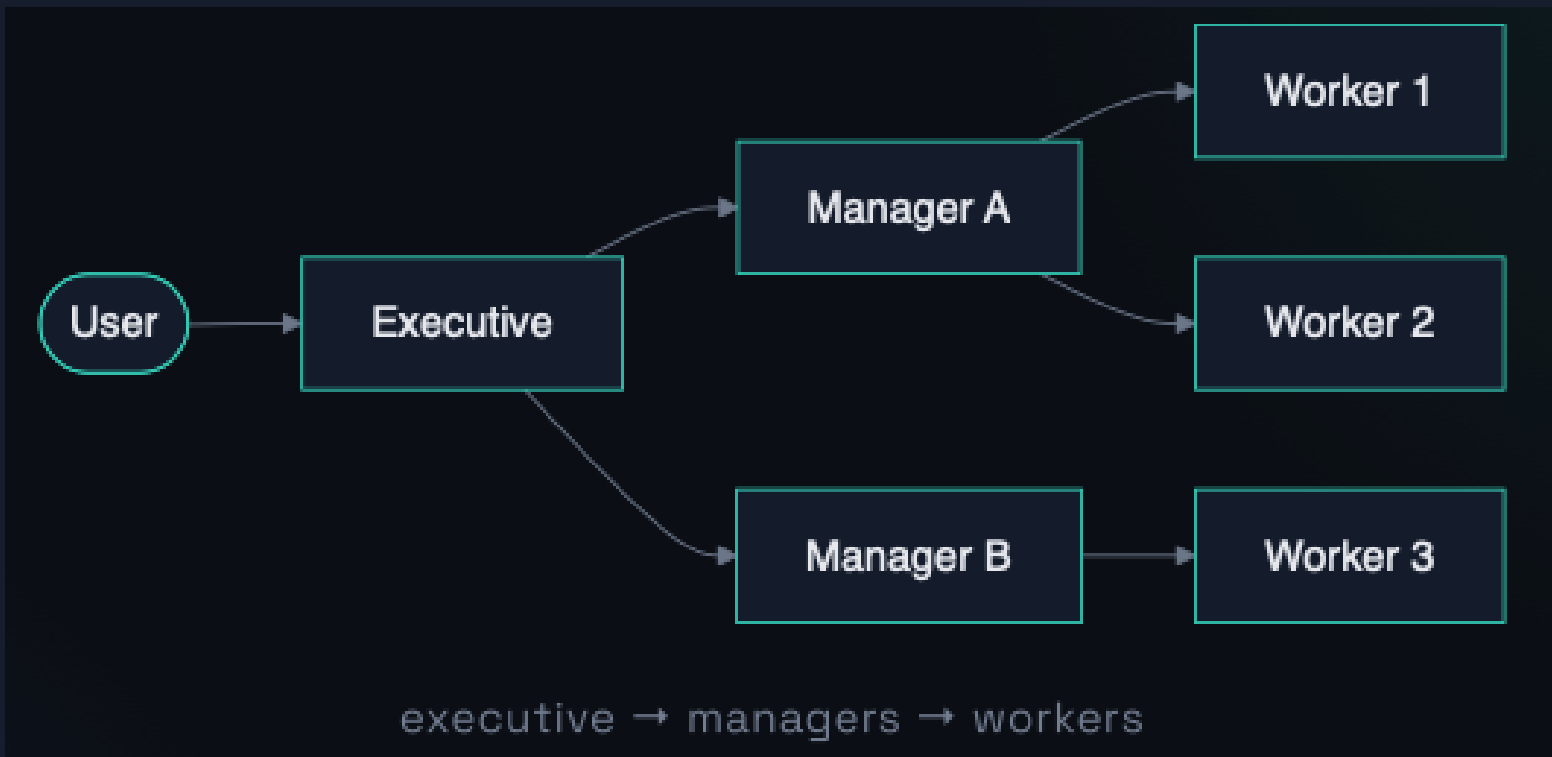
- **Exploratory or creative tasks** where the **path isn't known** upfront.
- Resilience matters more than predictability.



Watch out

- **Hard to debug** and **cost-control**.
- Context can explode
- Agents may **loop or duplicate work**.

Pattern C: Hierarchical



Use-cases

Software delivery / DevOps at scale

- Large-scale code migration/modernization
- End-to-end "ship this service" copilot

Research & analysis

- Deep research assistant
- Investment / equity research
- Competitive intelligence / due diligence

Content & knowledge production

- Marketing campaign production
- Book/course/report generation



When it fits

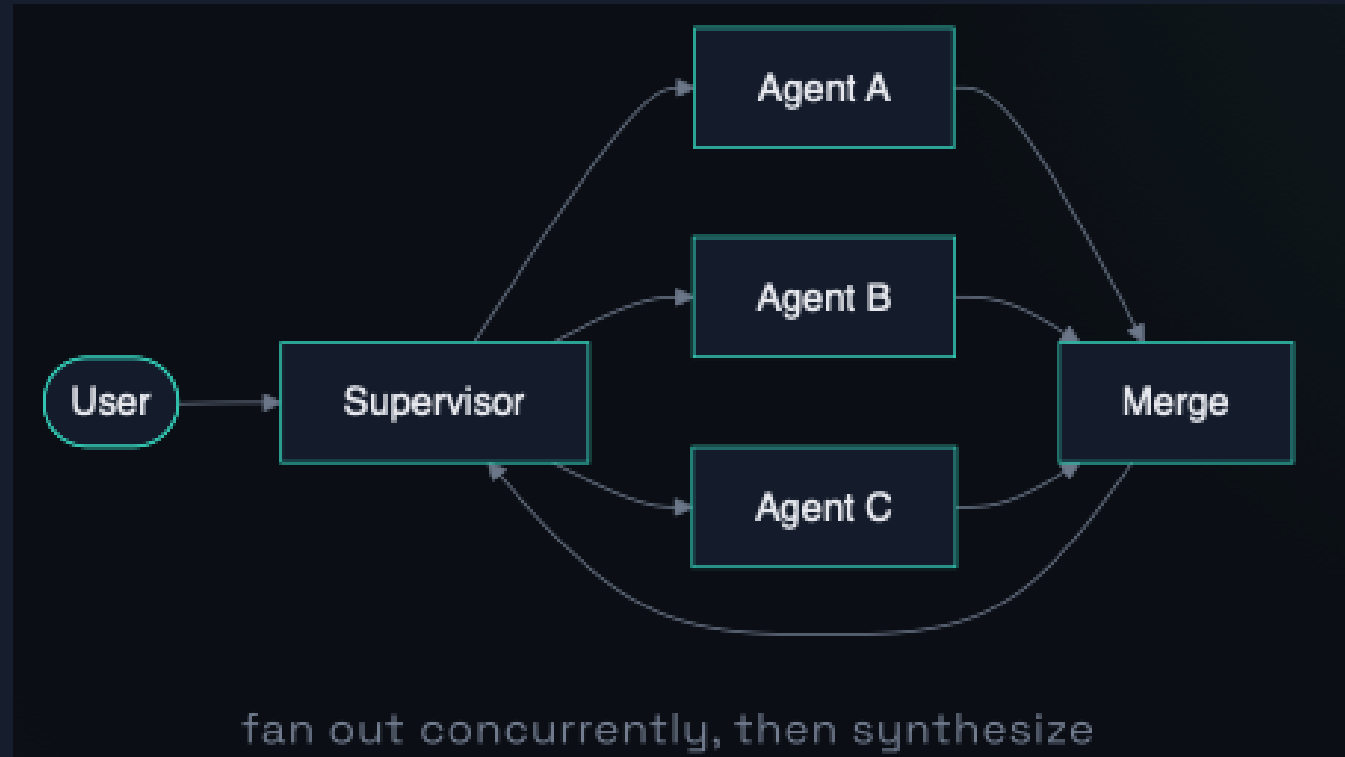
- Very **complex domains** with many **nested sub-domains**.
- Each level can use a different model tier.



Watch out

- Deep chains **multiply latency** — every level is an LLM call.
- Over-engineering for most cases.

Pattern D: Supervisor + parallel merge



Use-cases

Multi-dimensional evaluation of artifact

- Code review (parallel reviewers)
- Contract / document review
- Architecture / design review
- Loan / insurance underwriting

Fan-out research

- Market or competitive research
- Travel / itinerary planning
- Product / vendor comparison
- Grant/RFP or Literature scan

Consensus / ensemble for reliability

- Best-of-N generation
- Bulk triage/classification



When it fits

- Multi-faceted queries that **decompose into independent sub-problems** — research, analysis, aggregation.



Watch out

- Needs a non-trivial merge step.
- **Higher total cost** — every agent runs on every request.
- **No context sharing** → incompatible results

How to choose a pattern

Your Priority	Best fit	Avoid
Debuggability + clear ownership	Router	Swarm
Resilience / open-ended tasks	Swarm	Router
Many nested sub-domains	Hierarchical	Flat single agent
Speed on decomposable queries	Supervisor + parallel	Deep hierarchy
Low cost per request	Router	Supervisor (all run)

Strands Agents

Strands Agents is an **open-source SDK** for building agents using just a **few lines of code**



Ease of use: Intuitive agent development to get started in minutes instead of hours



Robust capabilities: Native tools, MCP servers, and natively integrated with AWS services

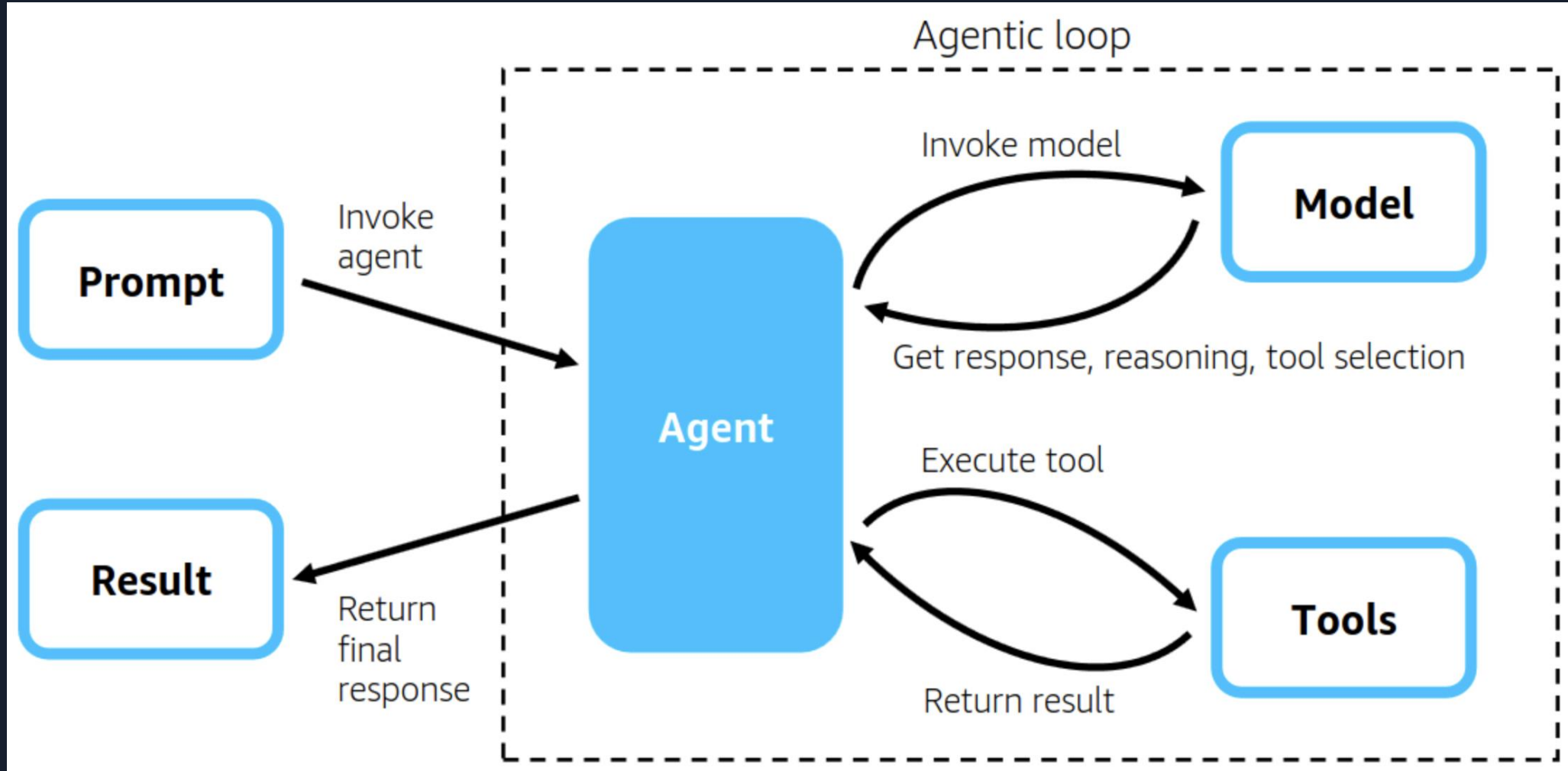


Extensibility: Support for custom model providers, custom tools, and MCP

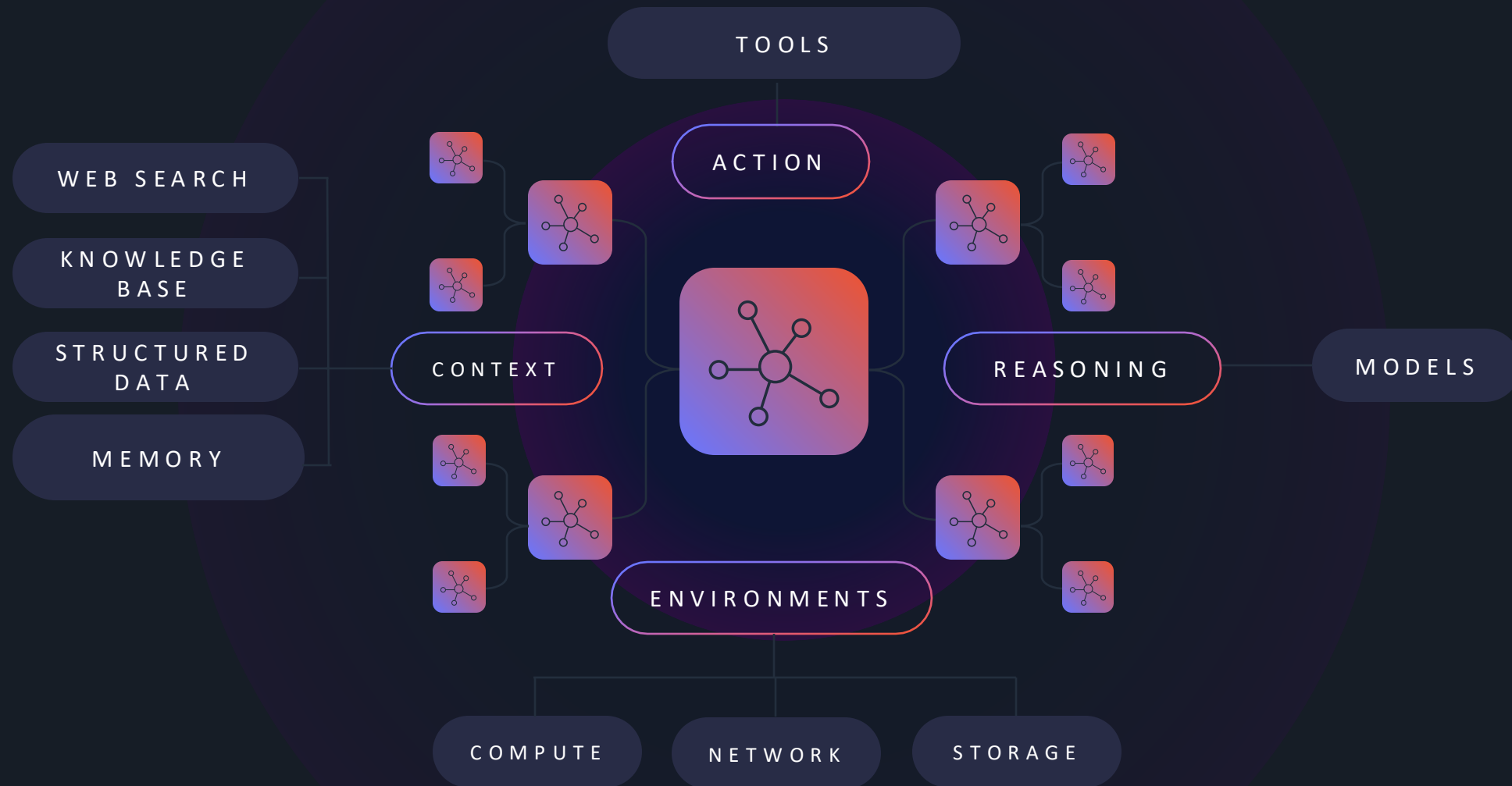


Rapid development: Quick prototyping and iteration

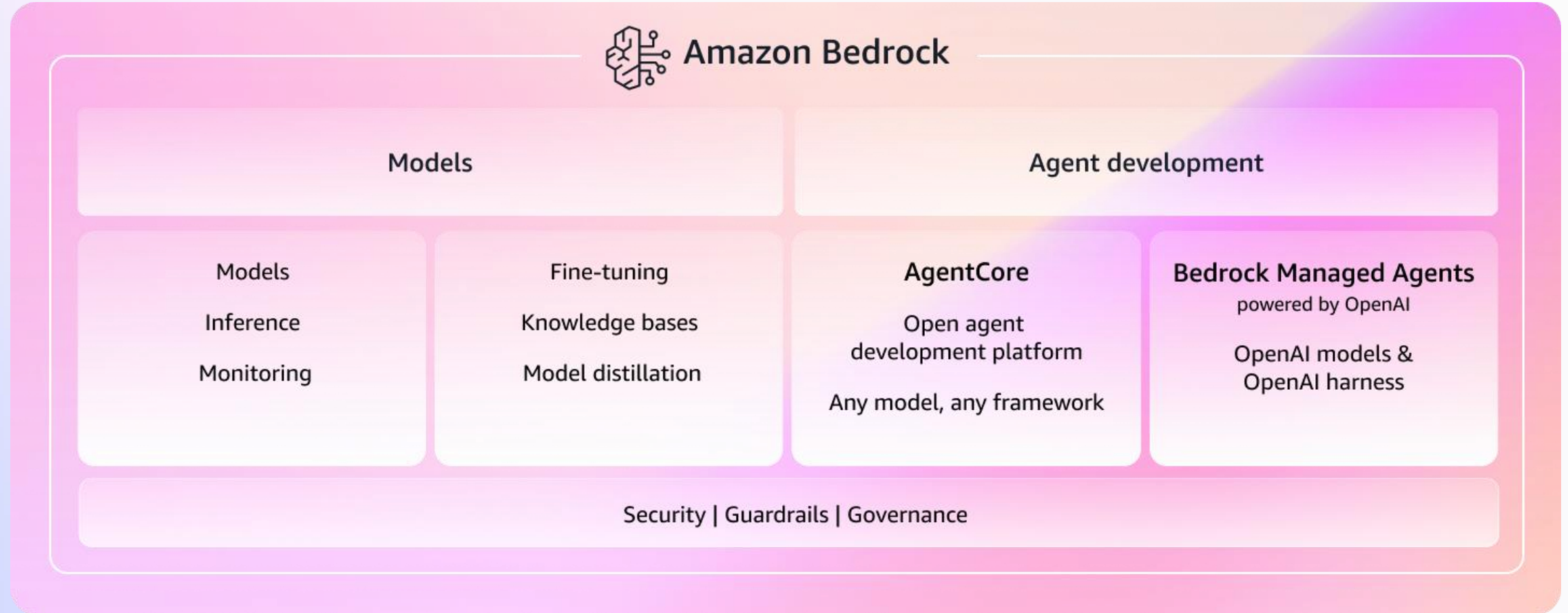
Strands: Simple agent loop built in



Agents need environments to run safely together



Amazon Bedrock - the platform for building generative AI applications and agents at production scale





Amazon Bedrock

Broad selection of fully managed models from leading AI companies

AI21labs

JAMBA

amazon

NOVA

ANTHROPIC

CLAUDE

cohere

COMMAND
EMBED
RERANK

deepseek

DEEPSEEK

Google

GEMMA

Luma

RAY

Meta

LLAMA

MINIMAX

MINIMAX

Mistral AI

MISTRAL
DEVSTRAL
MINISTRAL

Moonshot AI

KIMI

NVIDIA

NEMOTRON

OpenAI

GPT-5.5
GPT-5.4
GPT- OSS

Qwen

QWEN3

stability.ai

STABLE
DIFFUSION
STABLE IMAGE

TwelveLabs

MARENGO
PEGASUS

WRITER

PALMYRA

Z

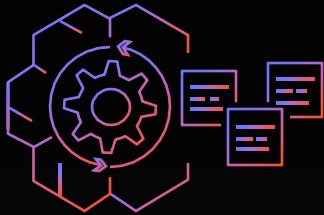
GLM



Amazon Bedrock AgentCore

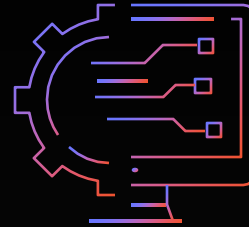
The platform to build, connect, and optimize agents. Any framework, any model.

SHIP AGENTS FAST



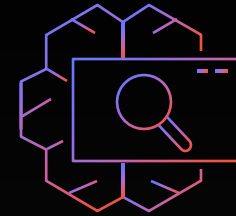
Build and deploy agents
without months of
infrastructure work

CONNECT TO ANYTHING



Give agents secure access to
MCP servers, APIs, and
knowledge bases

OPTIMIZE CONTINUOUSLY



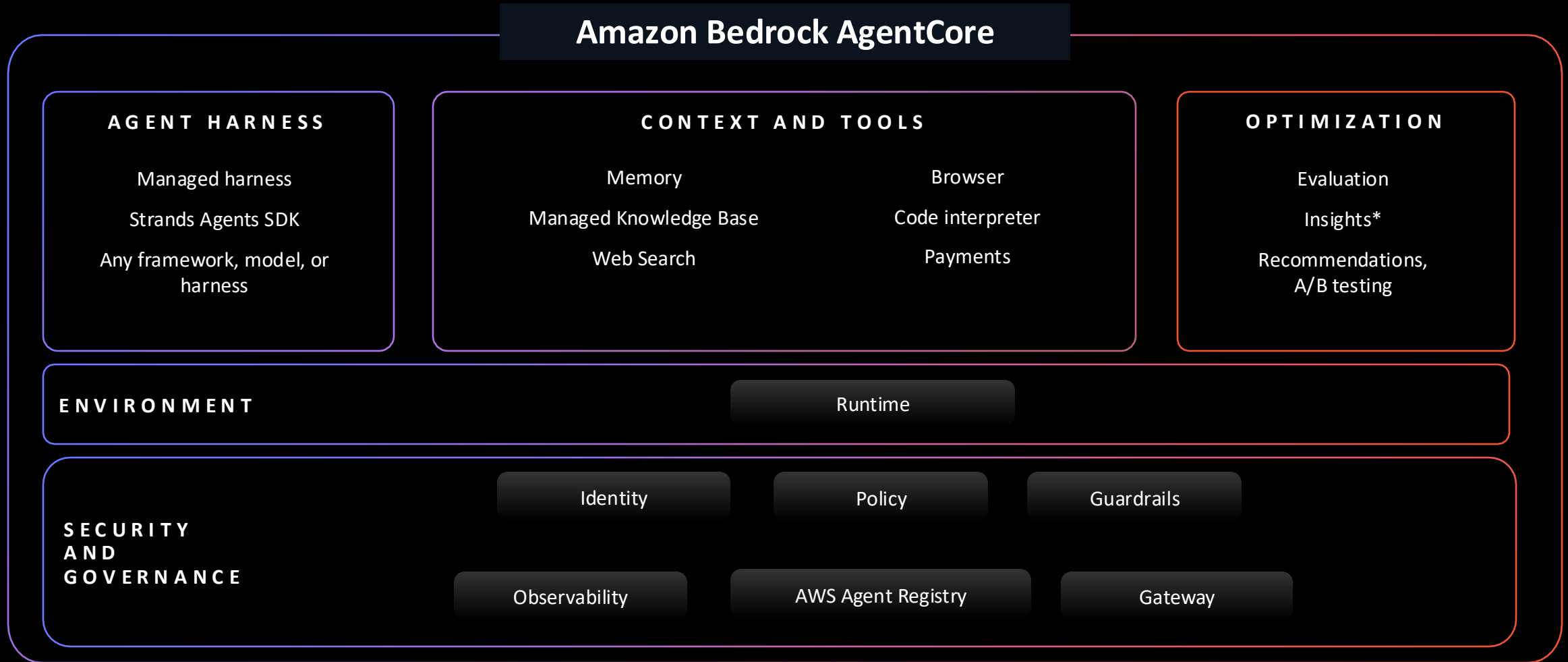
Trace every agent decision,
evaluate its performance, and
safely test improvements in
production

Security at scale

Enterprise readiness

Deterministic control

Everything you need for getting agents to production

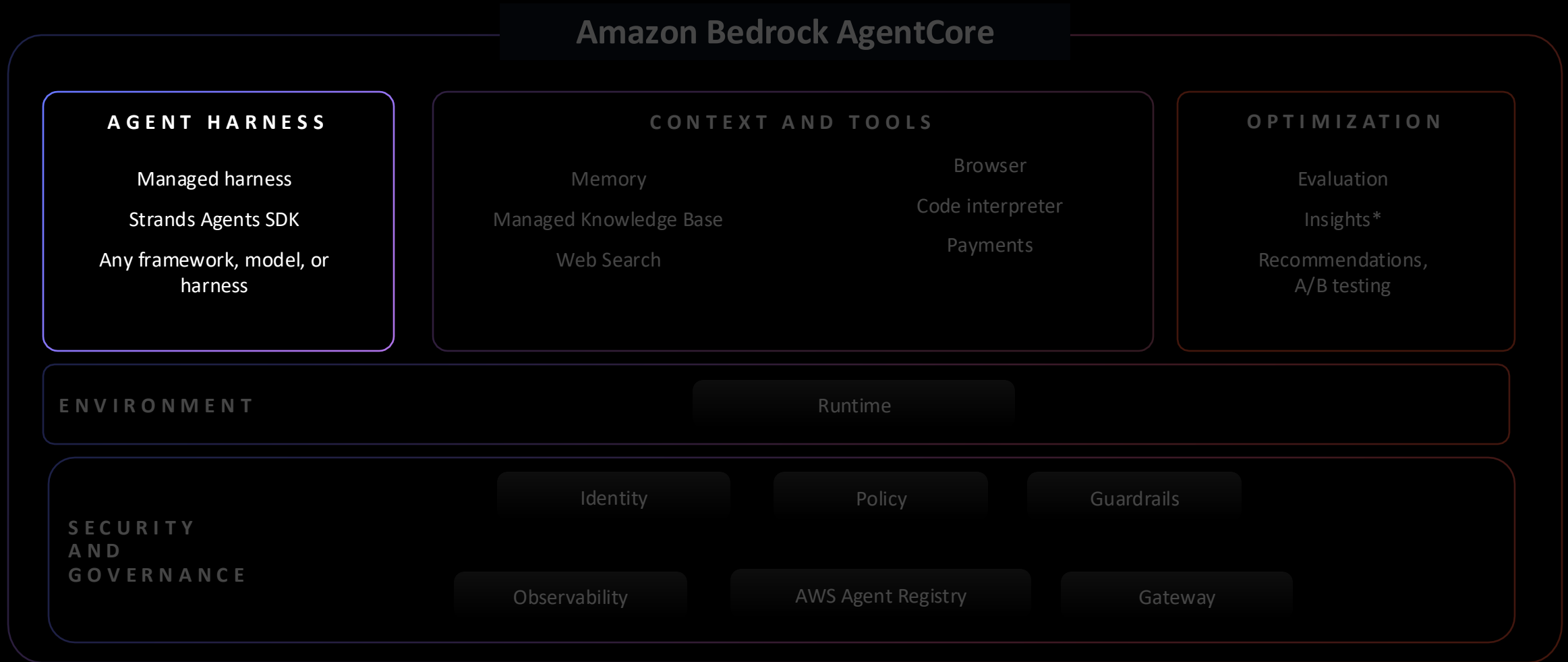


COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features



It starts with the agent harness



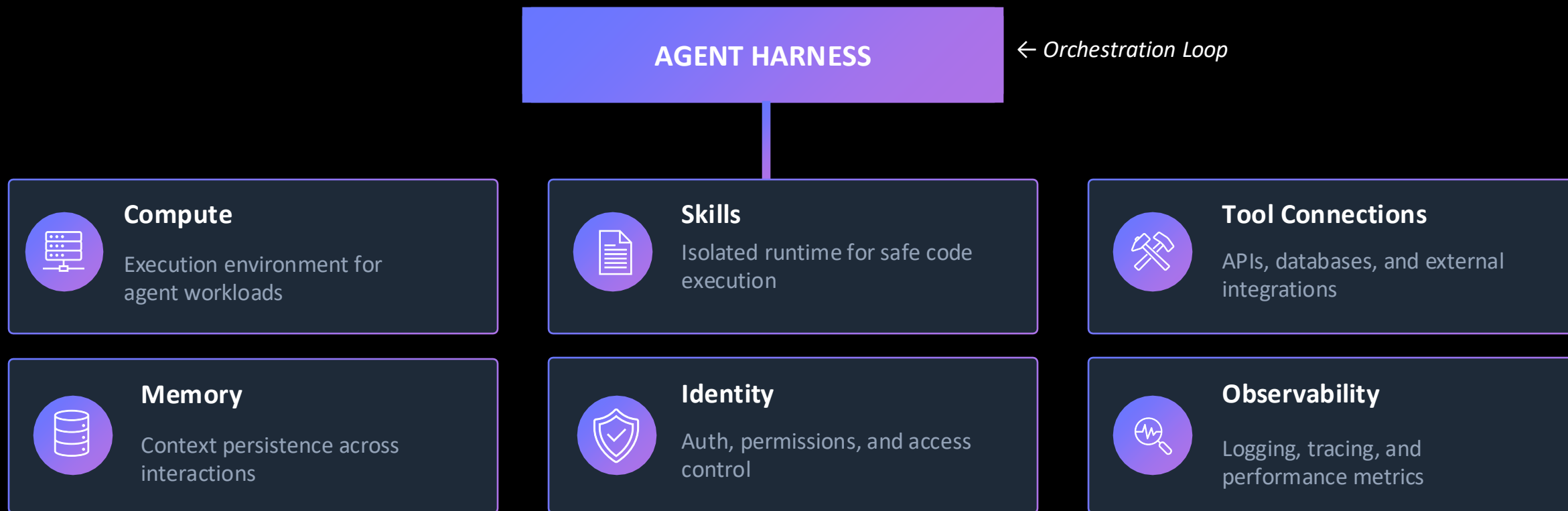
COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features

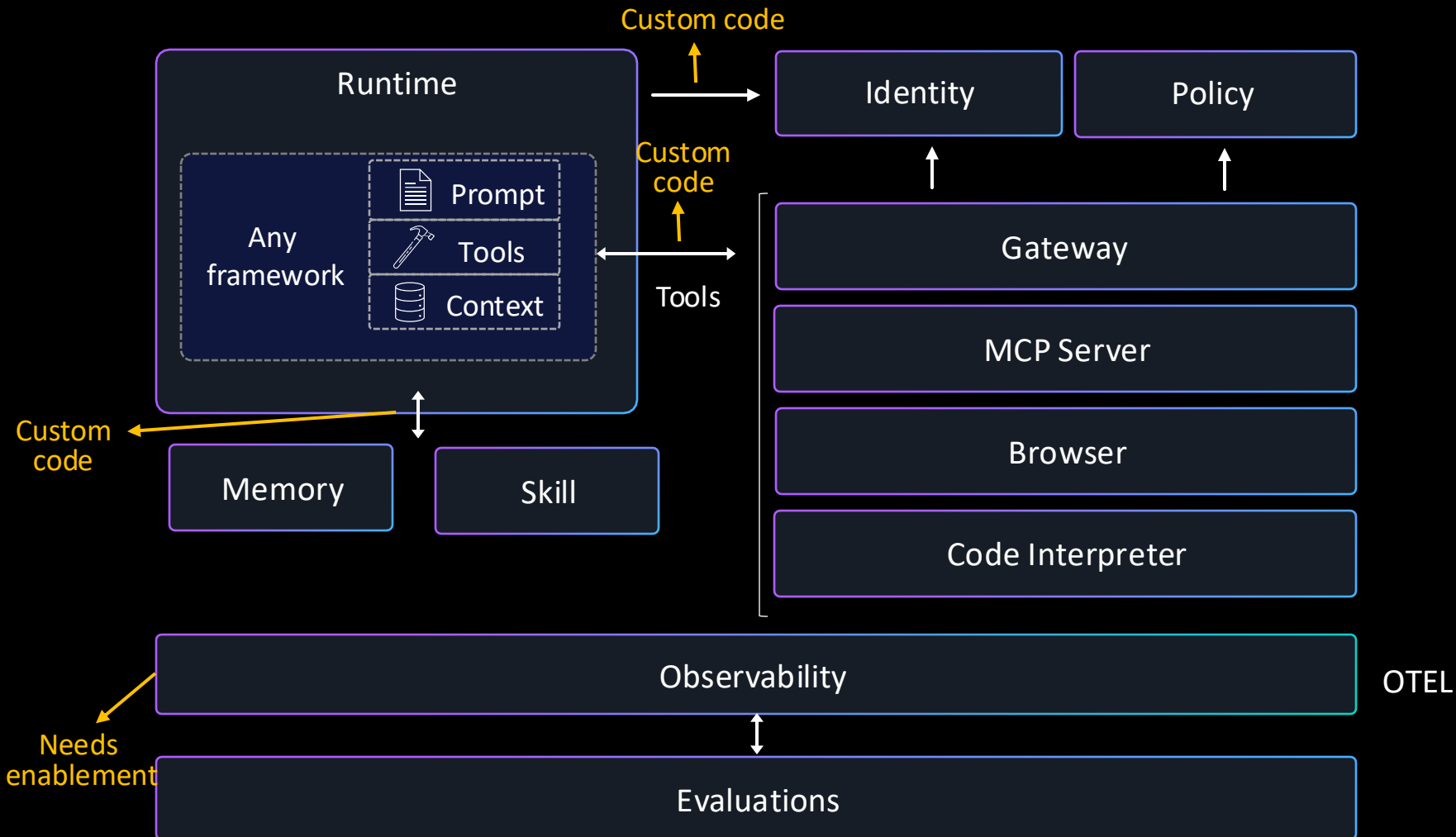


What is an agent harness?

The system that lets an agent actually run with orchestration loop + production infrastructure.

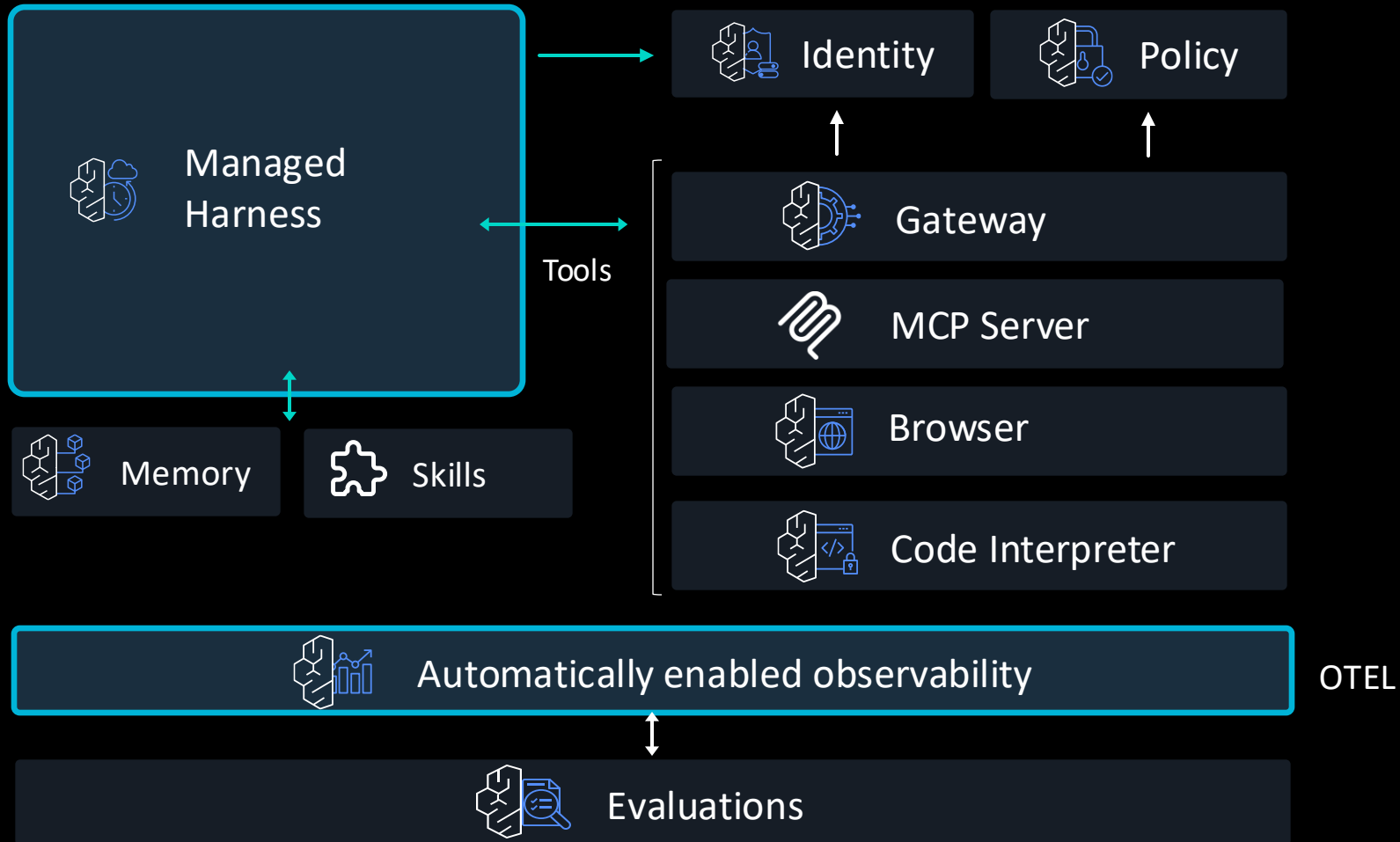


Building agents without a managed harness



When building agents with an open source agentic frameworks, a developer needs to connect the dots between the different agentic components. From the agentic framework to the deployment environment. Every new version of the agent needs to be redeployed and reconnected

Building agents with a managed harness



Managed harness on AgentCore handles this heavy lift for you and allows you to build agents as configurations. Integrate the different AgentCore capabilities as needed and update your agent dynamically. Export your harness to a framework configuration if you need more control

Managed harness in AgentCore

You bring the logic and we handle the infrastructure

YOU CONFIGURE

Models

Pick your model

Bedrock, OpenAI, Anthropic, or
bring any API

Tools

Attach your tools

MCP servers, gateway,
browser, code interpreter,
web search, functions

Memory & Skills

Plug in if needed

Short/long-term memory, AWS
skills, Git/S3 source,
container-baked

Identity & Evals

Connect & configure

OAuth, API keys, auth
Evaluators, A/B testing,
recommendations

WE MANAGE — INCLUDED AUTOMATICALLY

Runtime

Firecracker microVM

Observability

Traces, logs, metrics

Storage

Managed & persistent

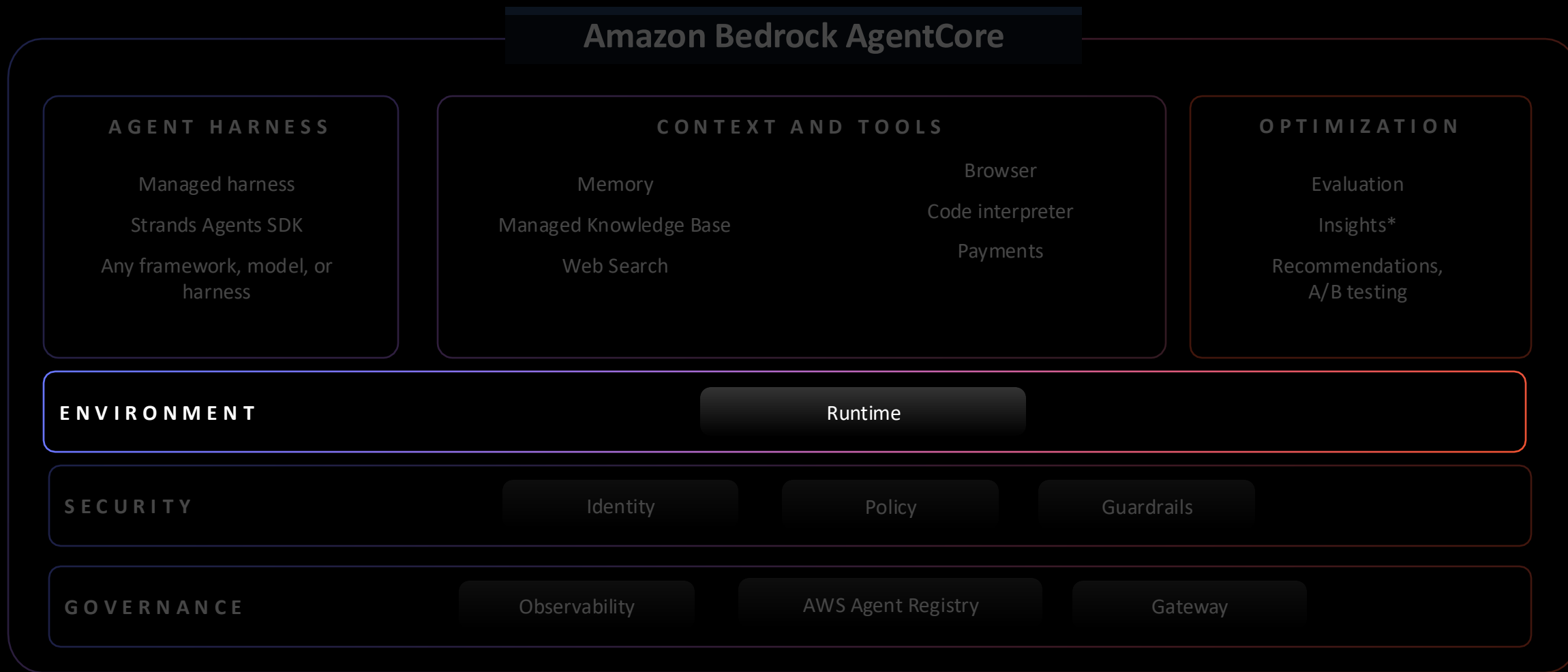
Security

Isolated by default

Zero infrastructure config. Just `CreateHarness` → `InvokeHarness`.



Where do agents run?

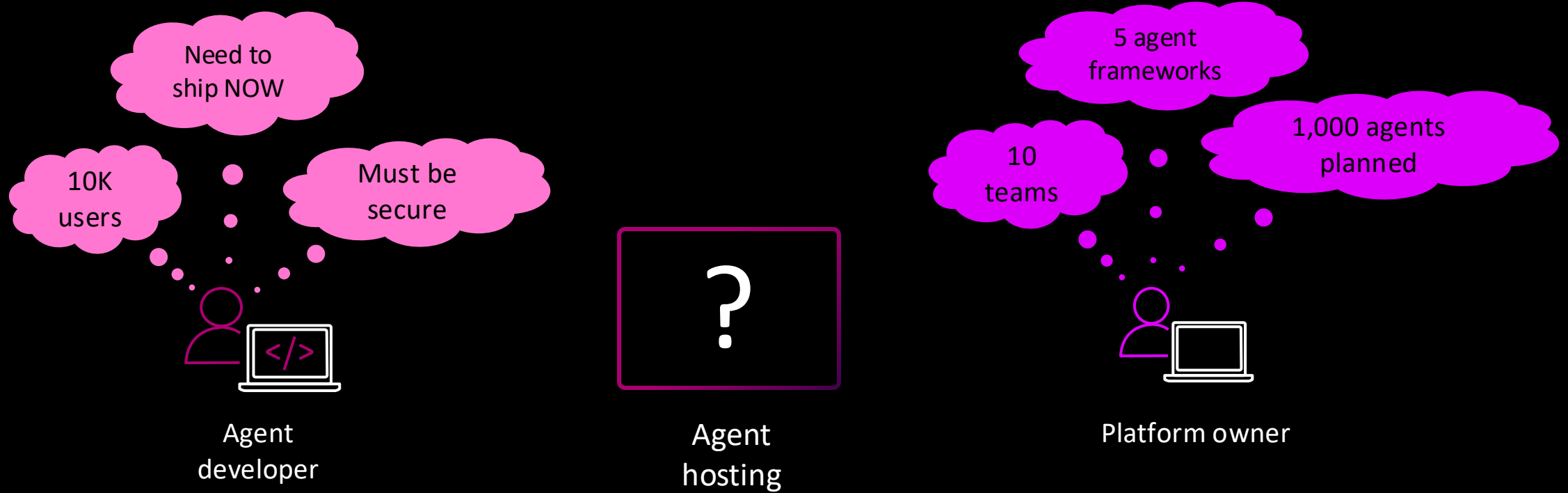


COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features

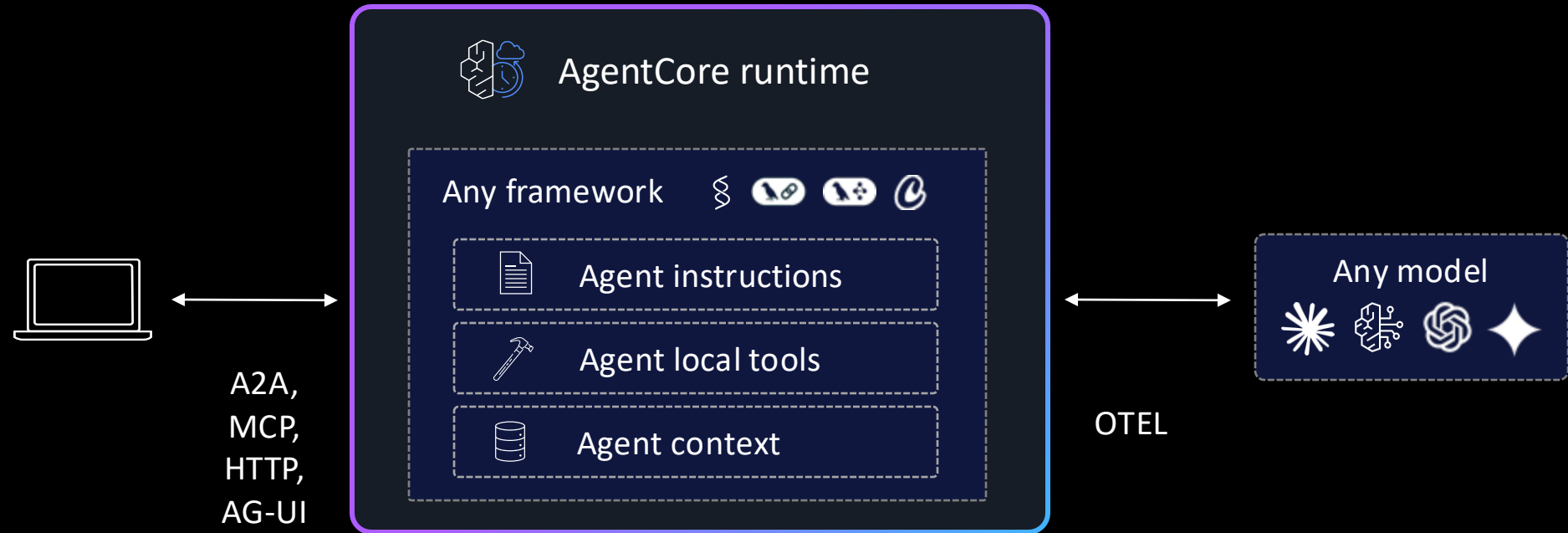


Agent hosting challenges





AgentCore runtime: from your laptop to planet scale, in one command



Time to value

Few lines of code, serverless, deploy in less than a minute

Secure

Session isolation, built-in auth, PrivateLink, VPC

Scalable

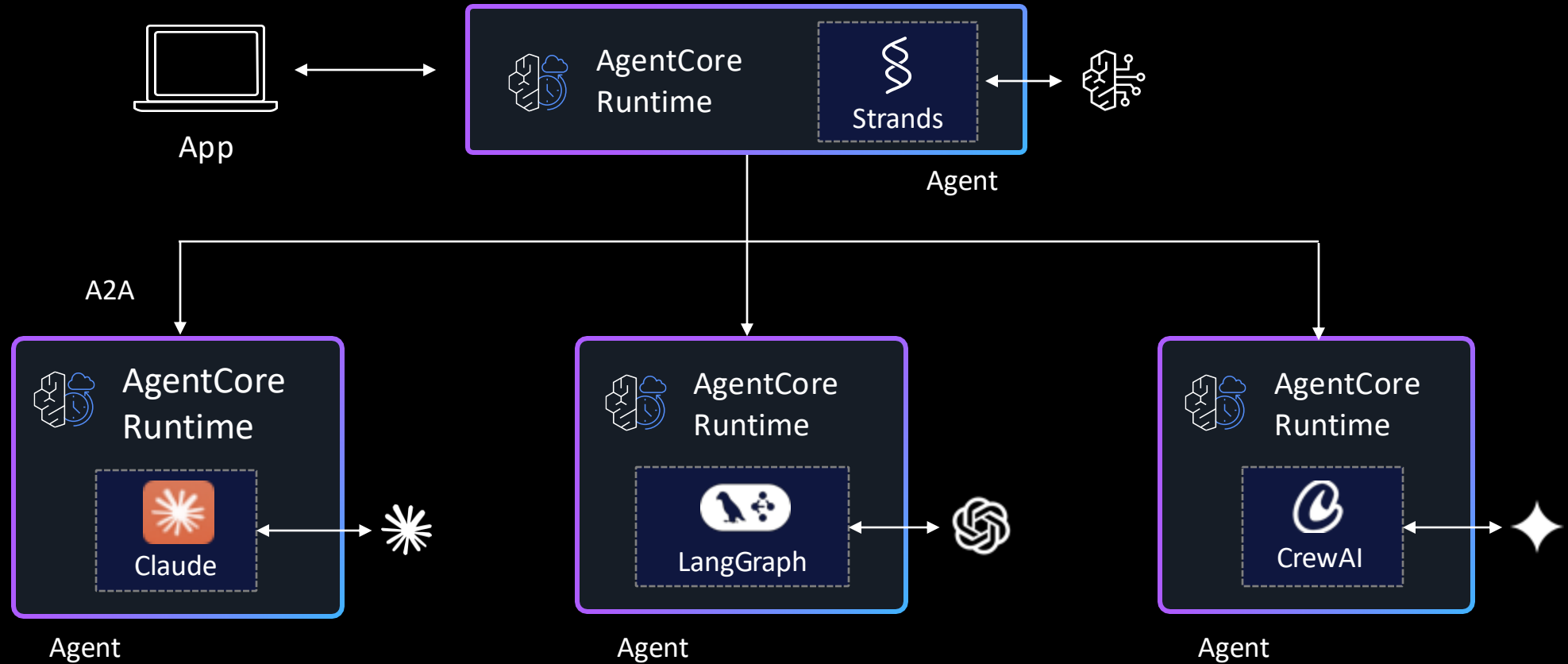
Large payloads, multi-hour workloads, async, bi-directional streaming

Cost performant

Pay for CPU utilization and memory allocation

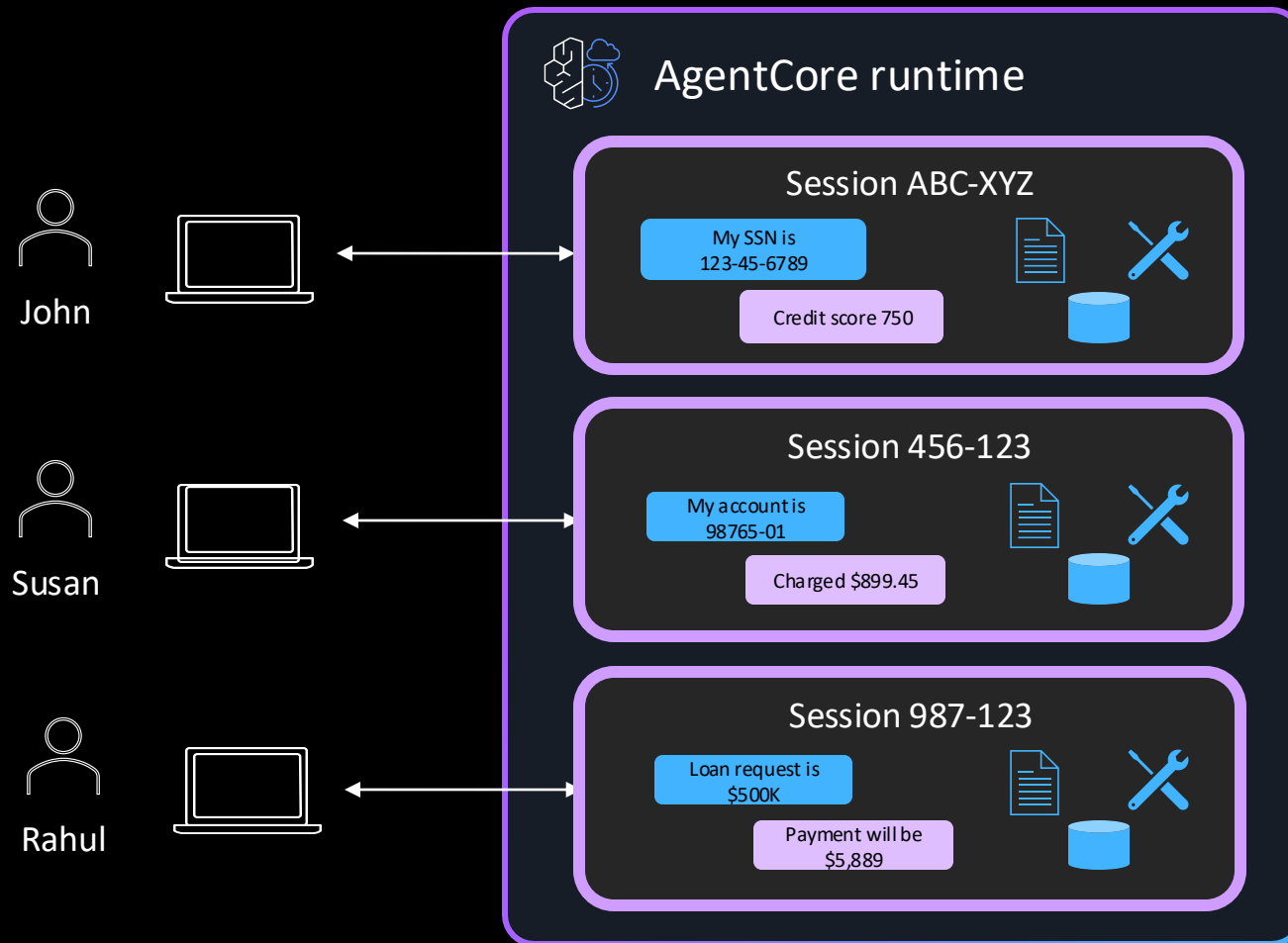


AgentCore runtime: deploy with any framework, any model





AgentCore runtime: every session in its own sealed box



Safely execute conversations

Dedicated microVM per session
(compute, memory, files)

Eliminates contamination risk

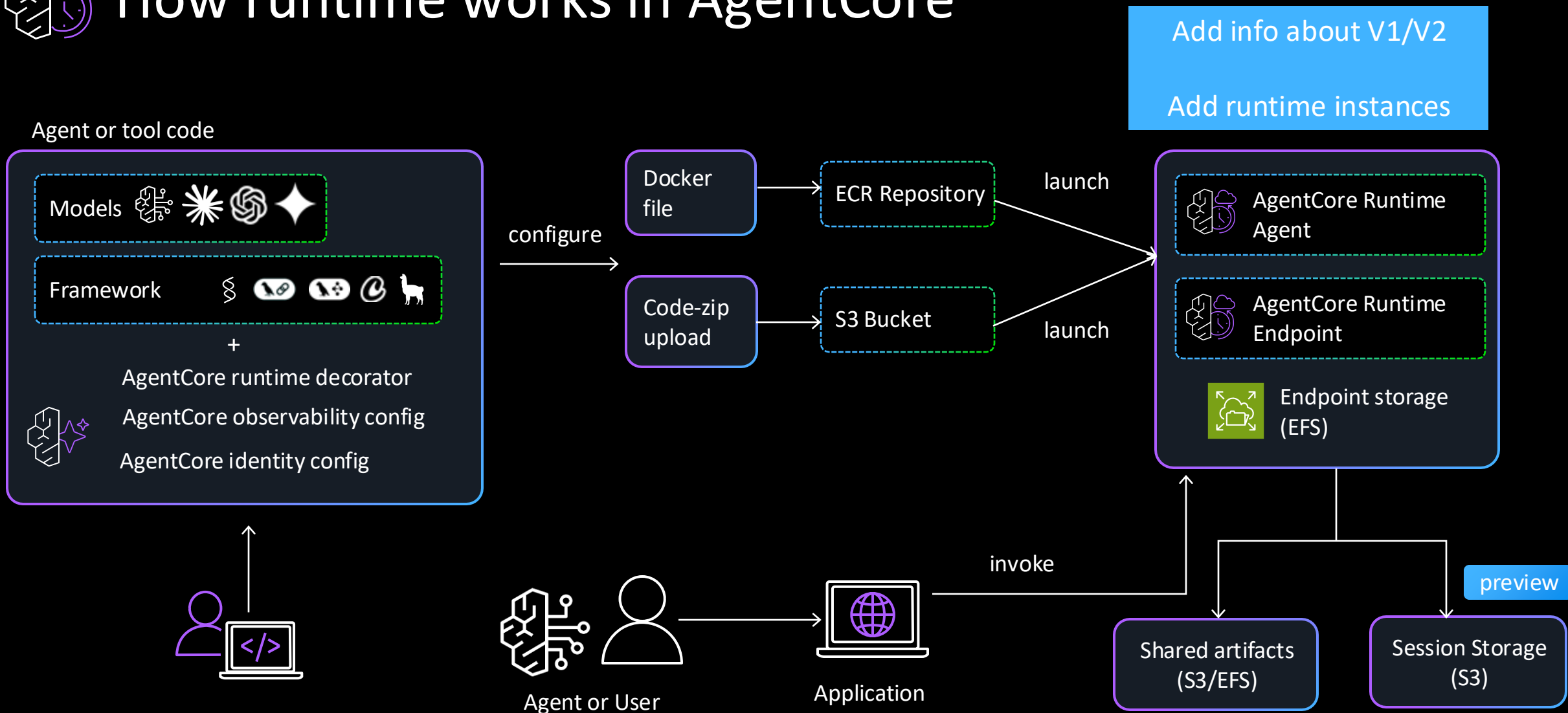
Prevents permission escalation

Deleted after session

Persist session state with filesystem
configuration



How runtime works in AgentCore



When to use which

Start with Managed Harness. Move to Runtime only when you need what it uniquely offers.

MANAGED HARNESS

**RECOMMENDED
DEFAULT**

Use when you want to...

- Get to production fastest
- Standard agent loop (reason → act → observe)
- Focus on prompts & tools, not infra
- Let AWS handle orchestration upgrades
- Single-agent workflows

Ideal for:

- Customer service bots
- Internal assistants
- RAG-powered Q&A
- Task automation agents
- Rapid prototyping to production

Export
(if needed)

AGENTCORE RUNTIME + FRAMEWORK

Use when you need to...

- Own the orchestration loop entirely
- Multi-agent collaboration patterns
- Graph or workflow-based architectures
- Bi-directional streaming (real-time)
- Custom hooks & middleware

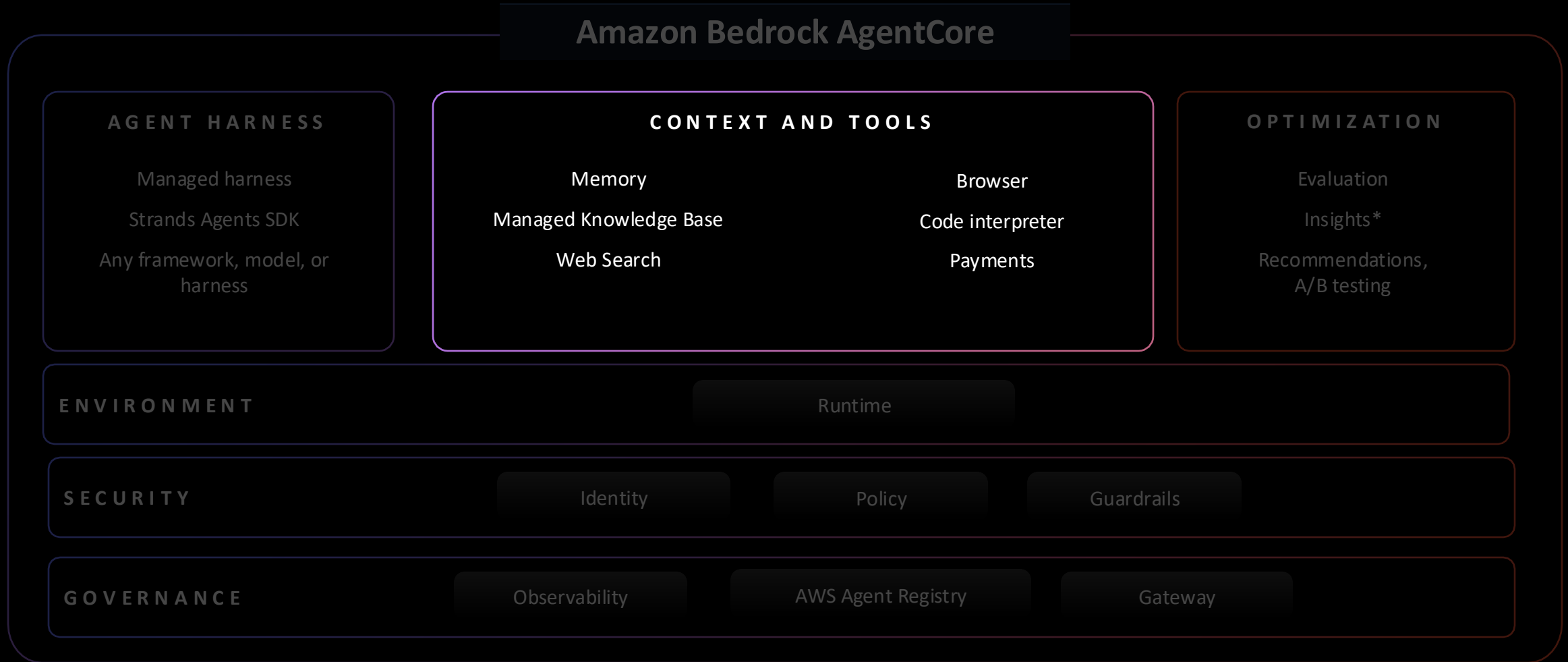
Ideal for:

- Complex multi-step orchestrations
- Real-time voice agents
- Custom frameworks (LangGraph, CrewAI)
- Coding agents

Managed Harness runs on AgentCore runtime under the hood and you can export your current configuration



What do agents know?



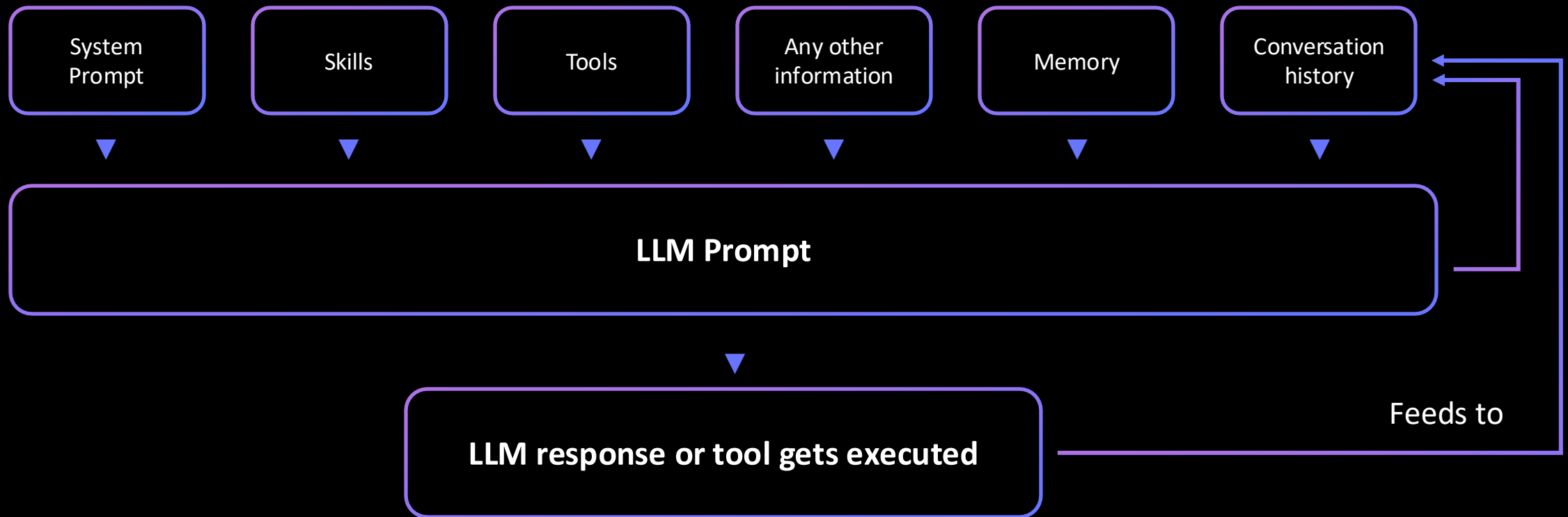
COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features



Context management: combining context sources into one prompt

A model knows the world. It does not know YOUR world



What's inside the LLM prompt?

Each turn, the agent assembles these context sources into a single prompt

System Prompt

You are a helpful coding assistant.
Follow best practices.
Be concise and accurate.

Tool Definitions

```
tools: [  
  {  
    name: "read_file",  
    params: {  
      path: string  
    },  
  },  
  {  
    name: "write_file",  
    params: {  
      path, content  
    }  
  }  
]
```

Skills

When writing Python:

- Use type hints
- Add docstrings
- Handle exceptions
- Follow PEP 8

Memory

Short term (conv history):

User prefers TypeScript.
Project uses React + Next.js.
Deployment: AWS ECS.
Last session: refactored auth module.

Long term (cross sessions):

User: "Add error handling to the API endpoint"
Assistant: "I'll wrap the handler in try/catch..."

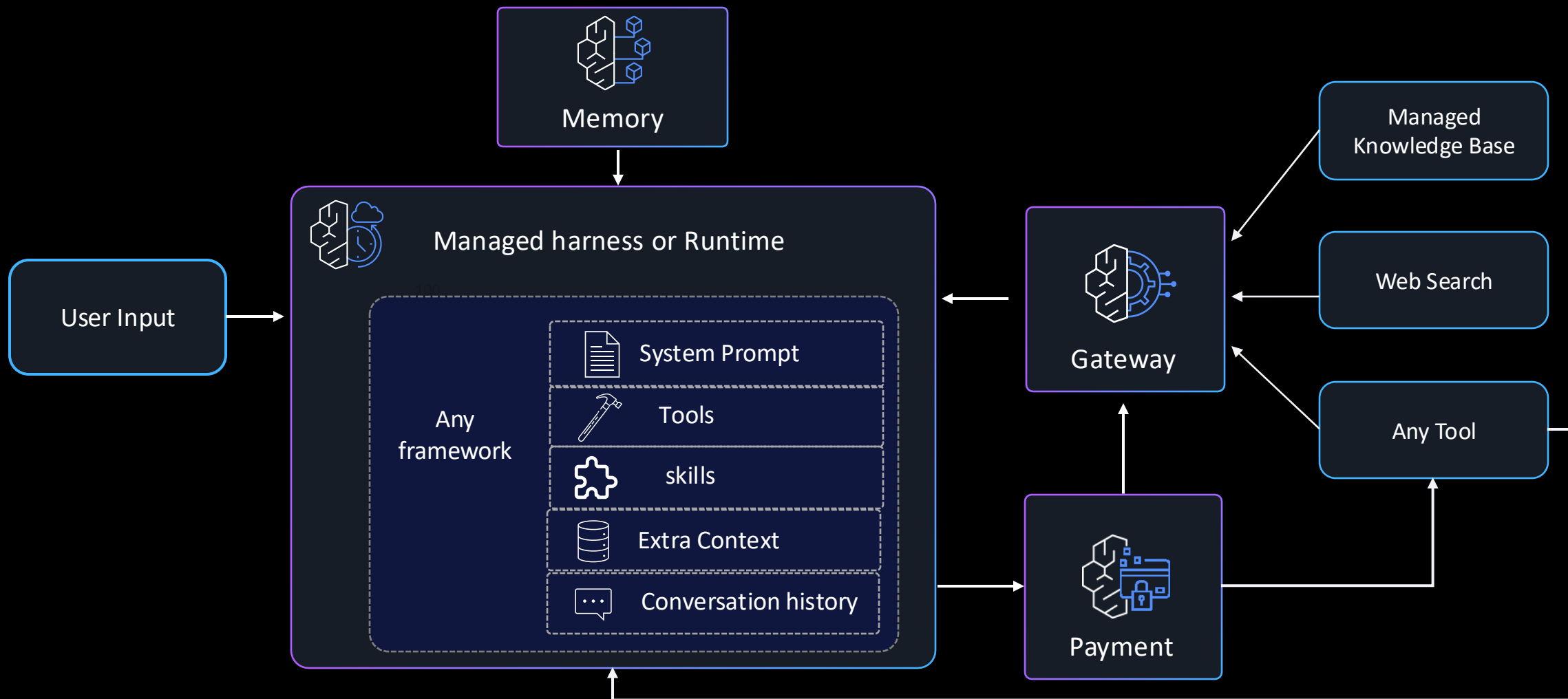
Other Information

Current date: 2026-08-24
Timezone: US/Pacific
Runtime: Node.js 20
AWS Region: us-west-2

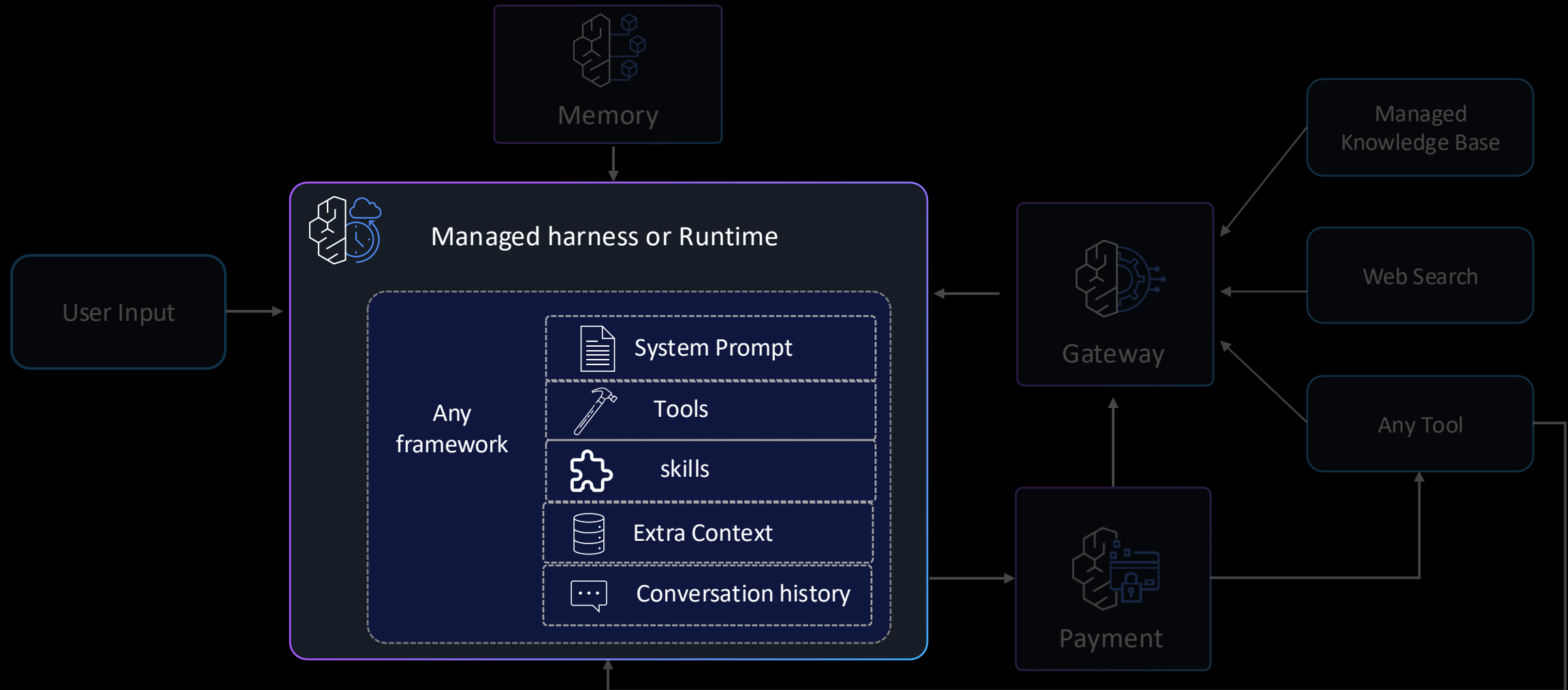
LLM Prompt

Managing agent context on AgentCore: end-to-end

From system prompts and conversation history to managed memory, tools, knowledge, and payments at scale



Orchestrating the context of your agent



Use any agentic framework or a fully managed harness to orchestrate your agent's context with AgentCore

BRING YOUR OWN

Any Agentic Framework + Runtime



- Use Strands Agents, LangGraph, CrewAI, or any framework to orchestrate your system prompt, skills, tools, conversation history and other information to manage your context
- Full control over orchestration logic

OR

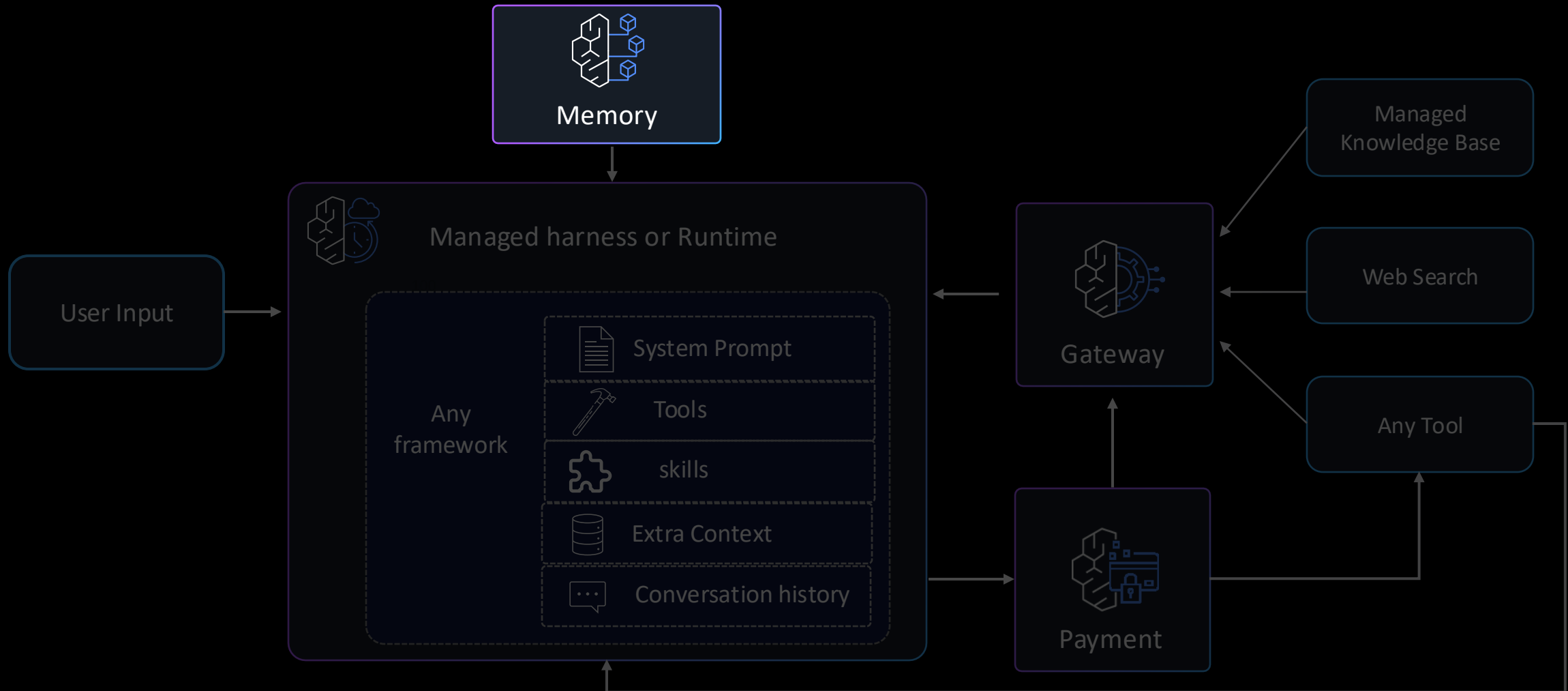
FULLY MANAGED

Managed Harness



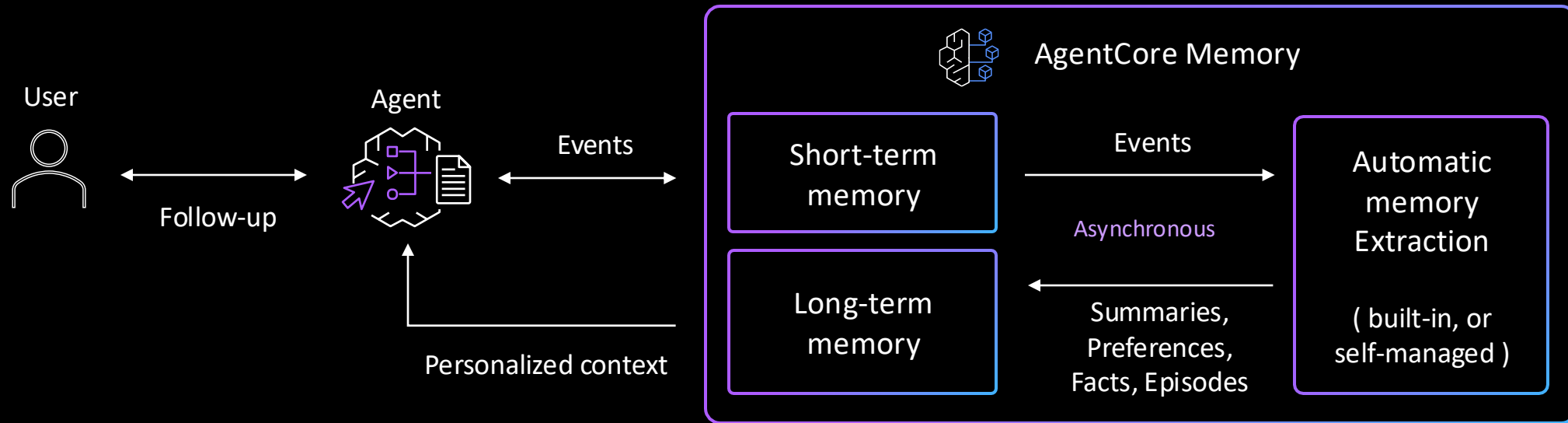
- Provide the system prompt
- Connect skills
- Connect tools
- Everything else is managed for you

Cross session knowledge





AgentCore memory: Give agents a memory that outlasts the conversation



Flexible extraction

Session summaries, user preferences,
and custom strategy pipelines

Serverless

Minimal setup,
no infra management

Secure

Encryption at rest and in transit,
access control

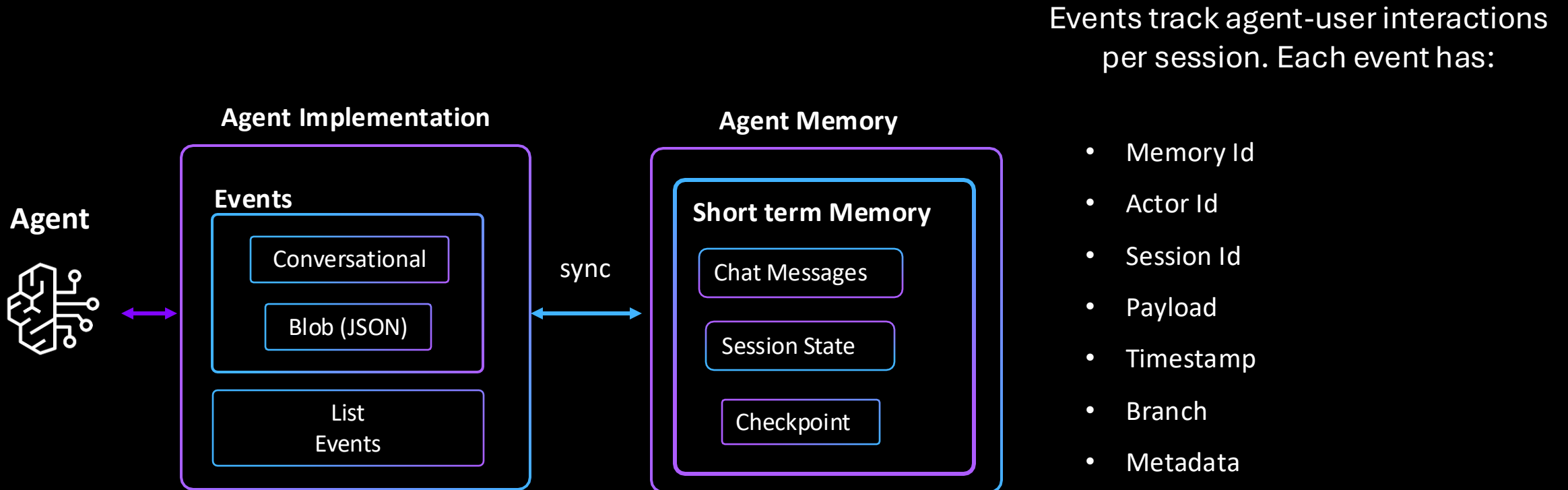
Flexible namespaces

Organize and share memories

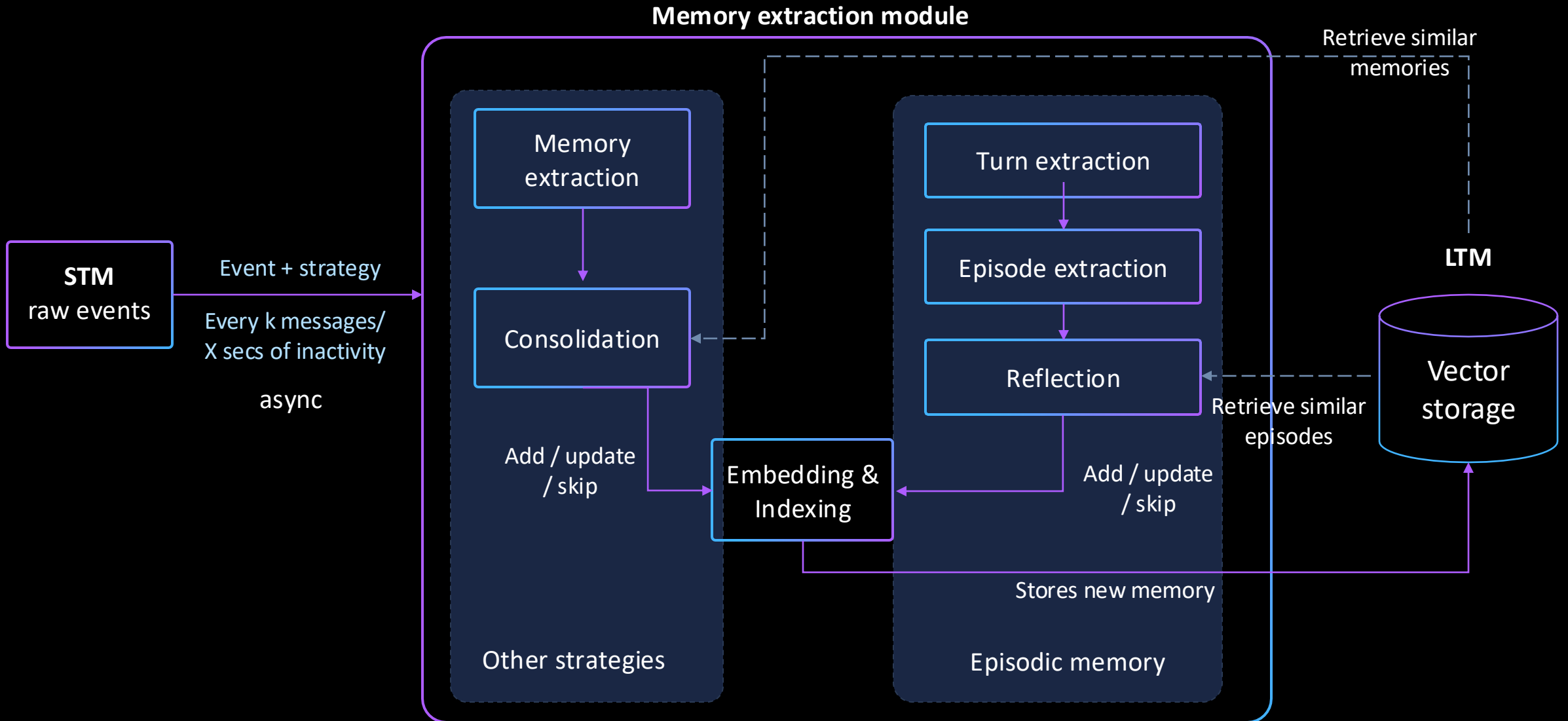
Plugs into agent frameworks

LangGraph, Strands, ...

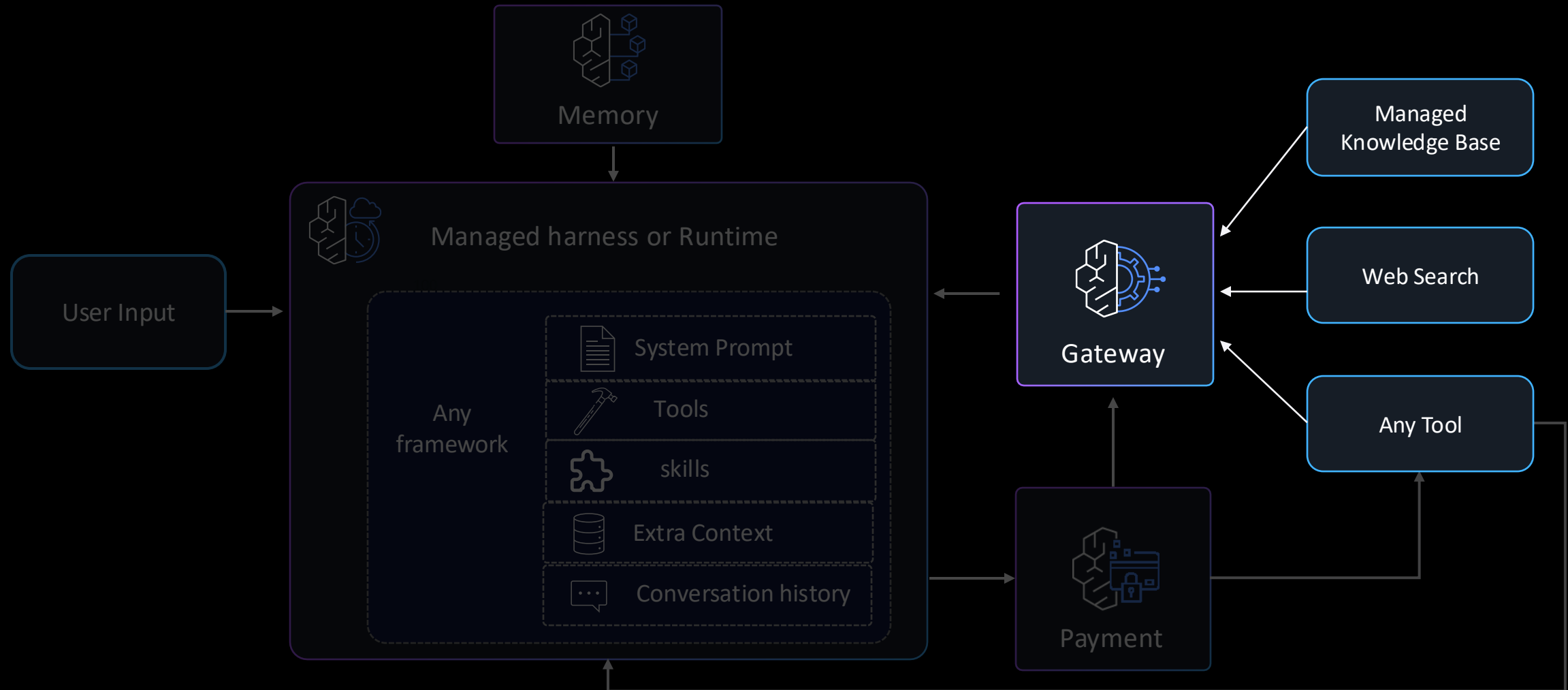
AgentCore Memory – Short-term memory (STM)



AgentCore Memory - Long-term memory (LTM)

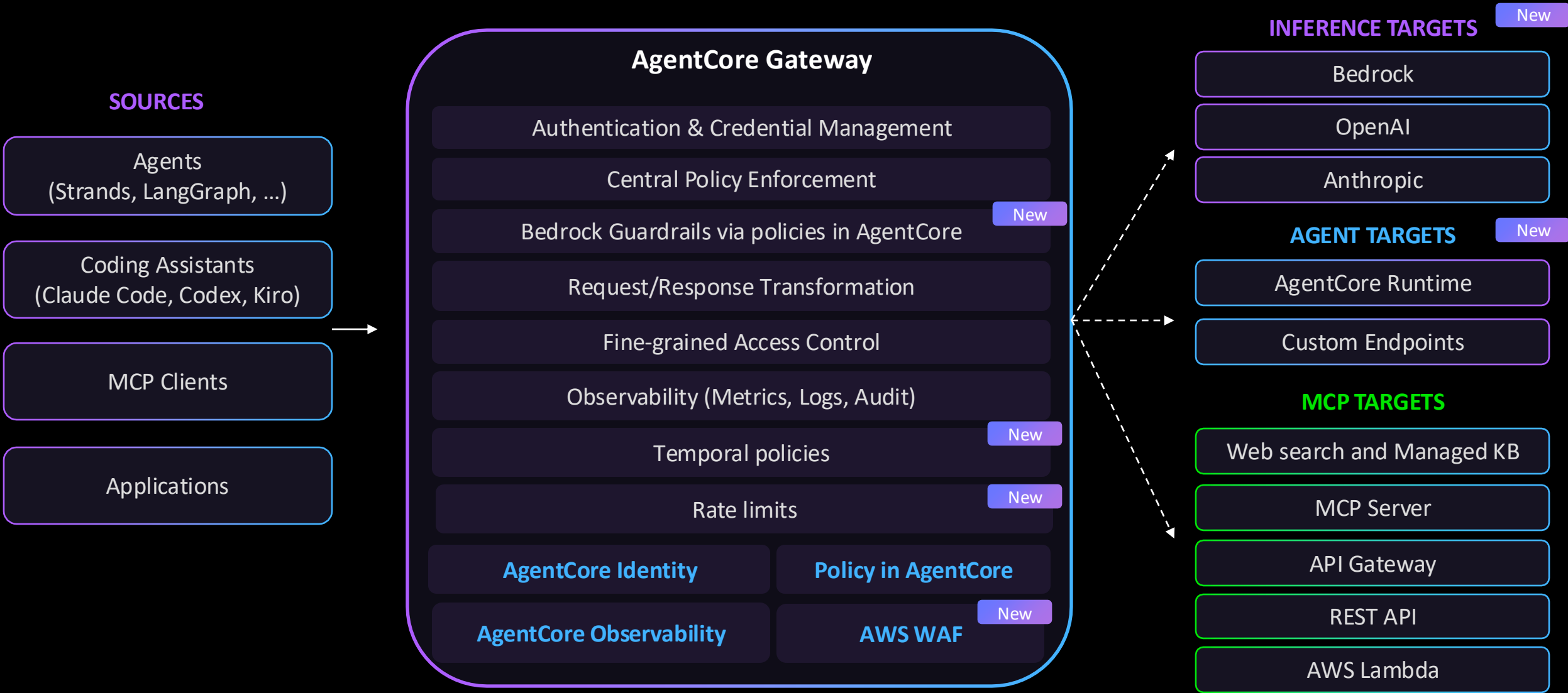


Using tools to interact with live systems



AgentCore Gateway: Serverless AI Gateway

Connect and govern agents, tools, and models through a single secure endpoint



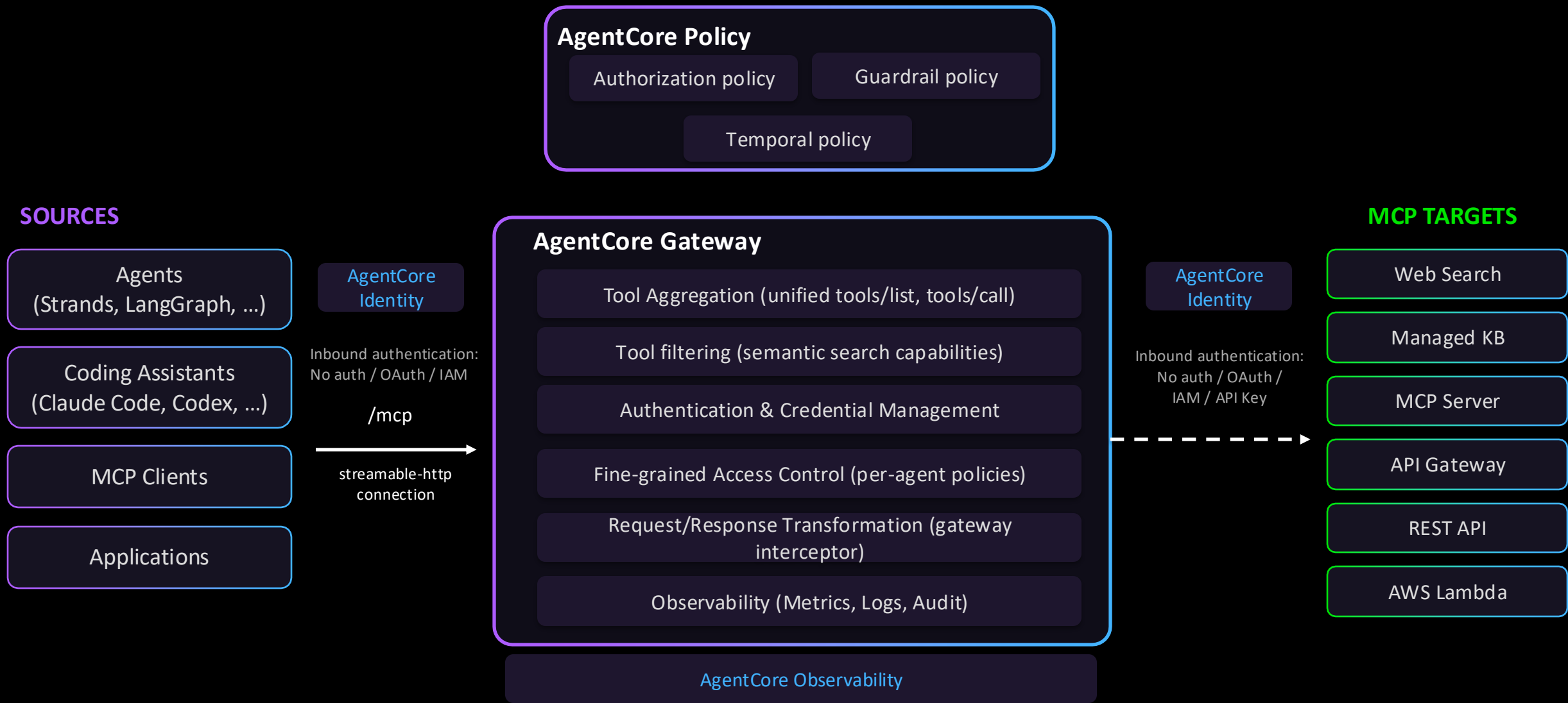
AgentCore Gateway: Serverless AI Gateway

Connect and govern agents, tools, and models through a single secure endpoint



Gateway as your agent's tool layer

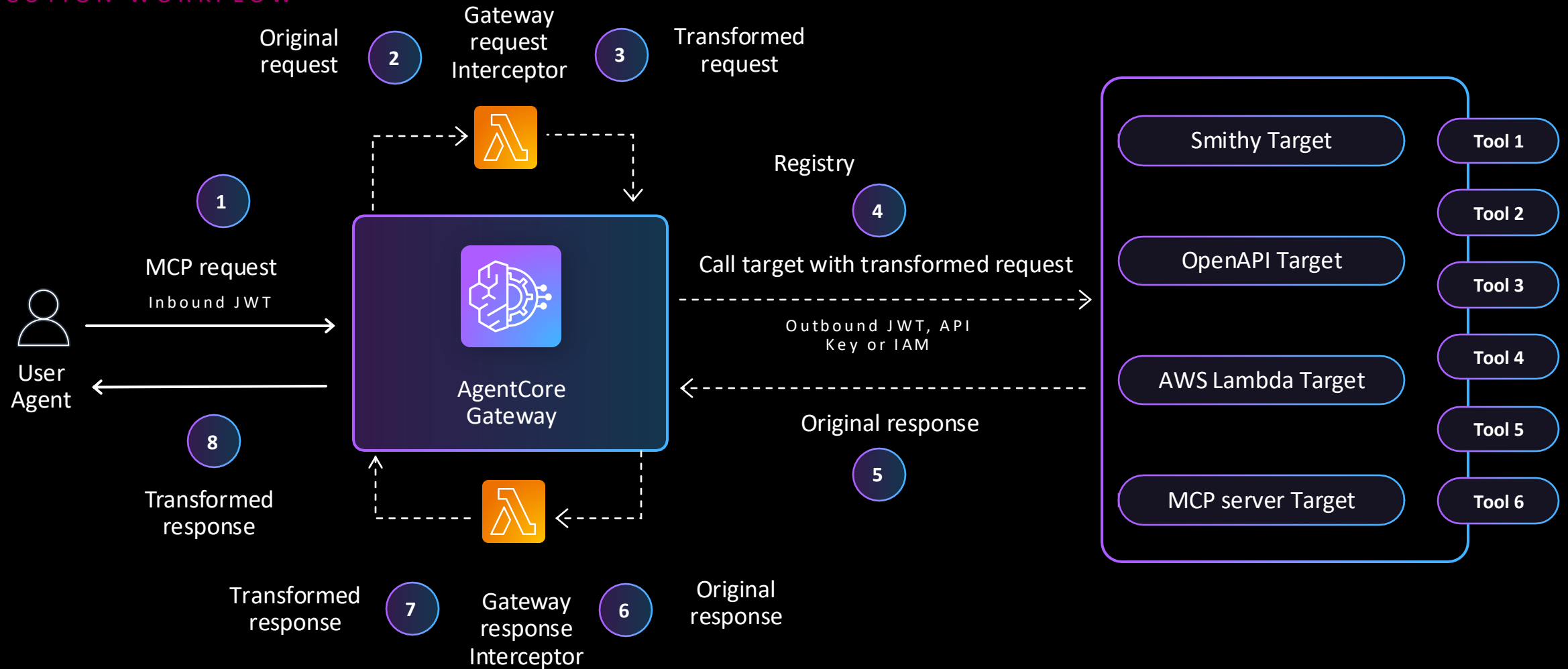
One MCP connection gives your agent access to all tools, with built-in auth, governance, and observability



Gateway Interceptors

Transforming requests and responses without changing the tool implementation

EXECUTION WORKFLOW



Web Search on AgentCore: give your agents current, accurate web answers

Web search under the hood

Web Index

Tens of billions of docs,
including long-tail content

Real-Time Updates

Index refreshed
continuously; new content in
minutes

Privacy & Security

All queries within AWS
infrastructure; no data
egress

Proprietary web index and knowledge graph

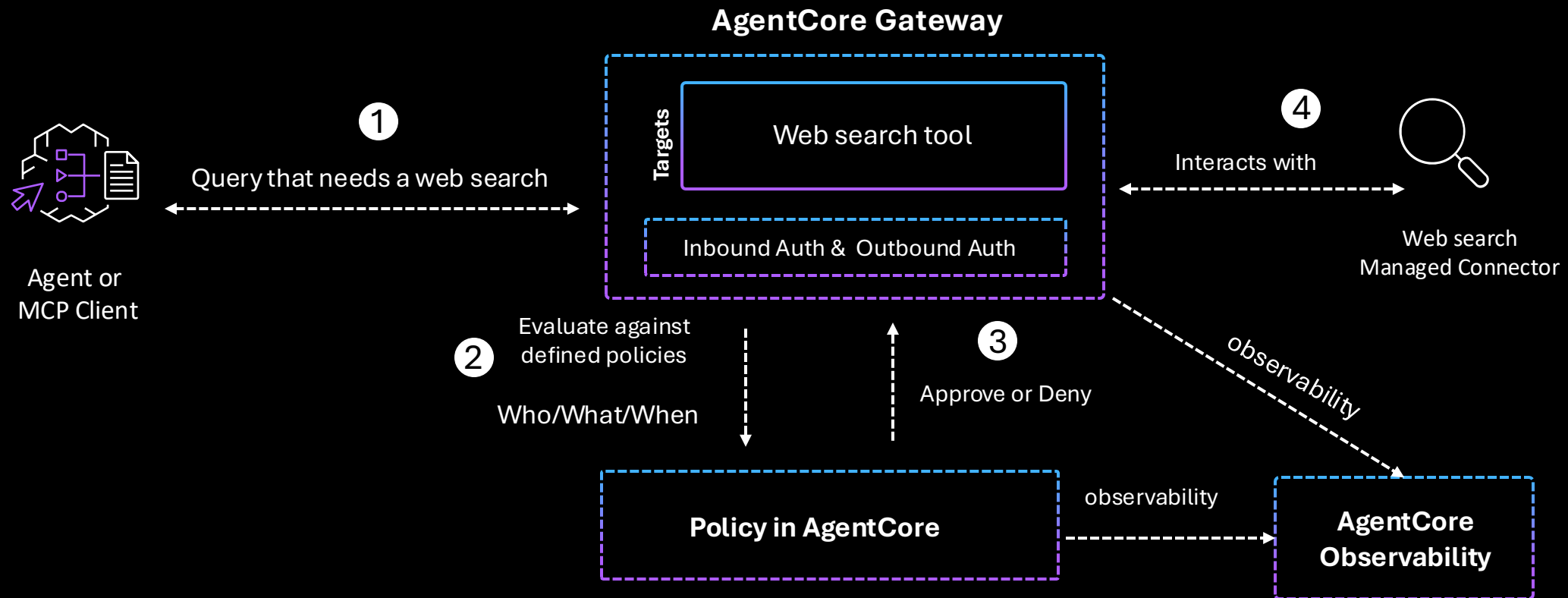
Latency optimized for real time applications

High retrieval quality

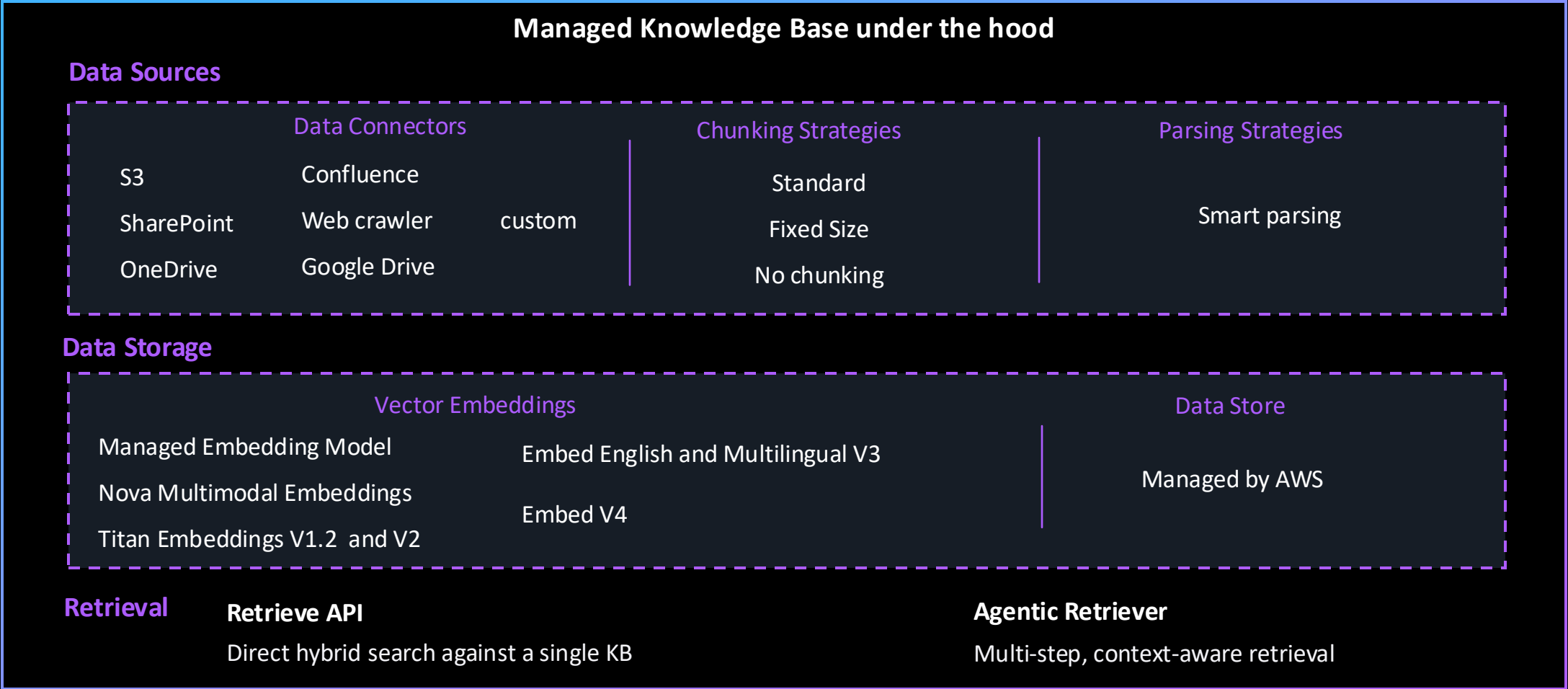
Configurable retrieval controls

How Web Search works in AgentCore

MCP compatible endpoint accessible over HTTP with IAM auth

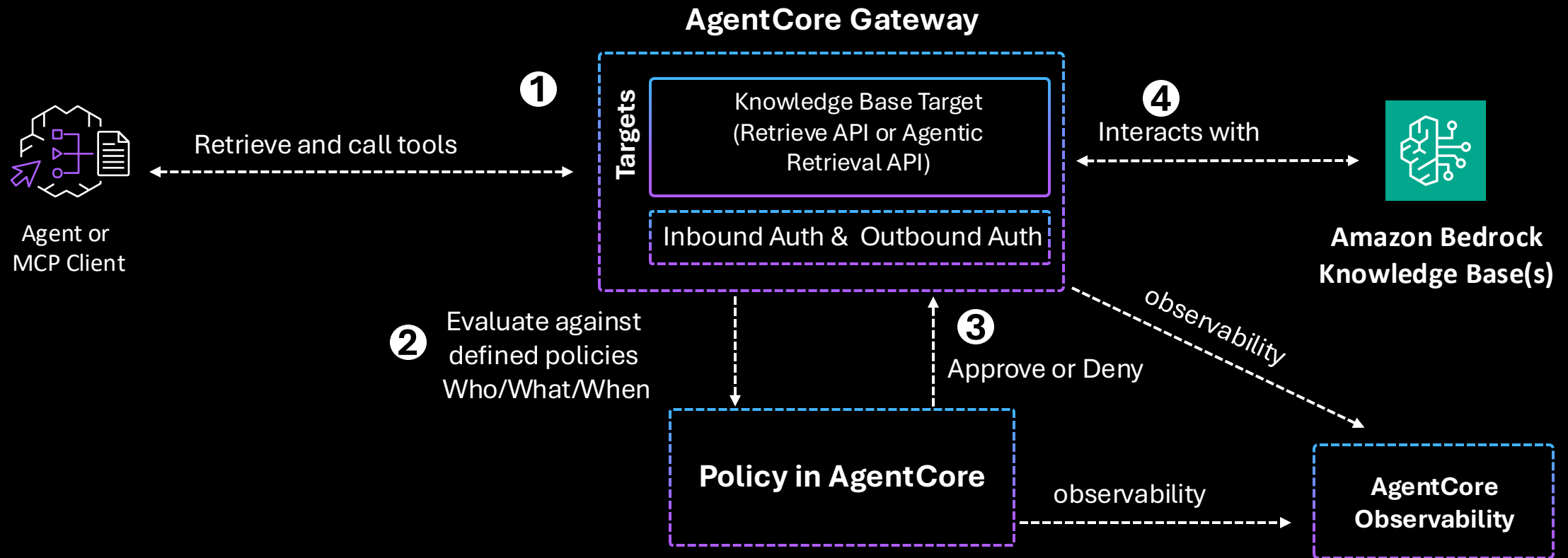


Bedrock Managed Knowledge Base: ground agents in the truth of your business



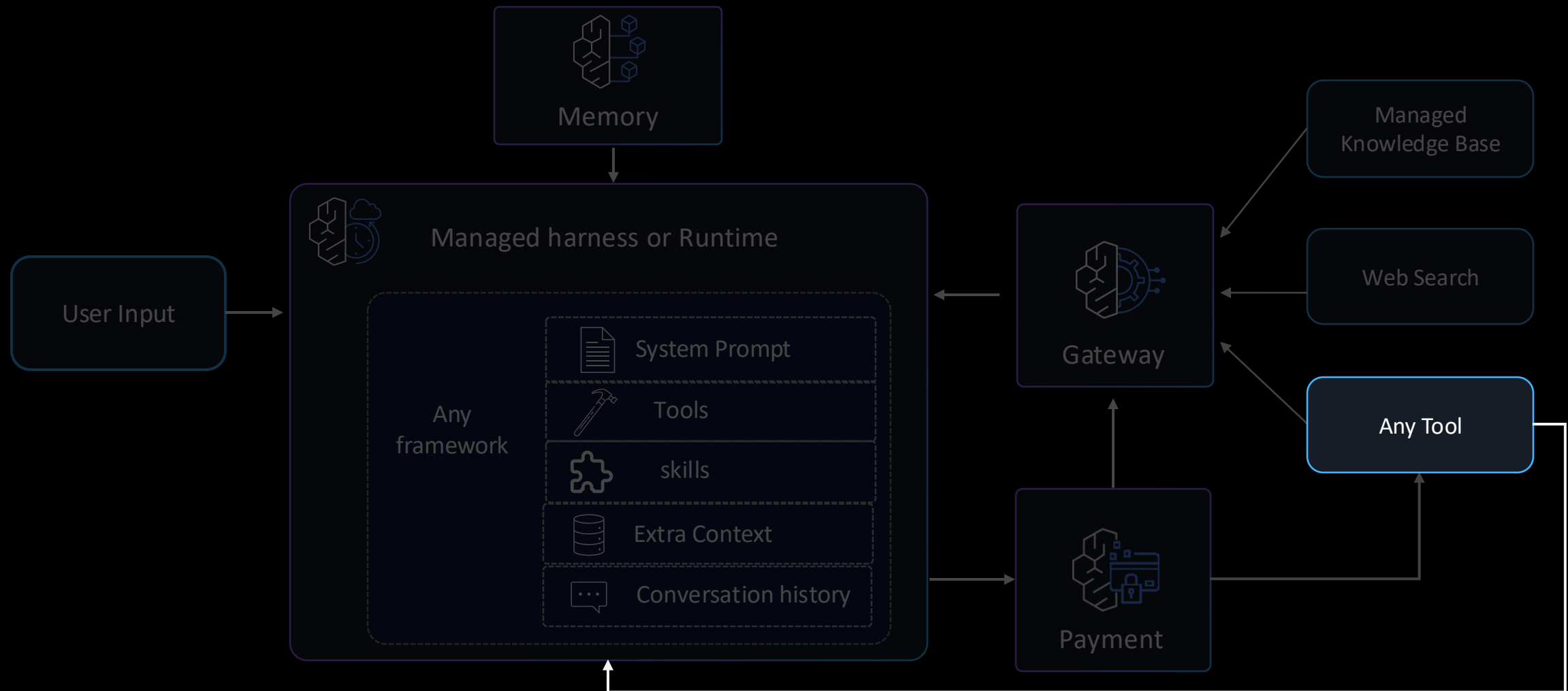
How Managed Knowledge Base works in AgentCore

Native gateway integration with built-in policy and observability

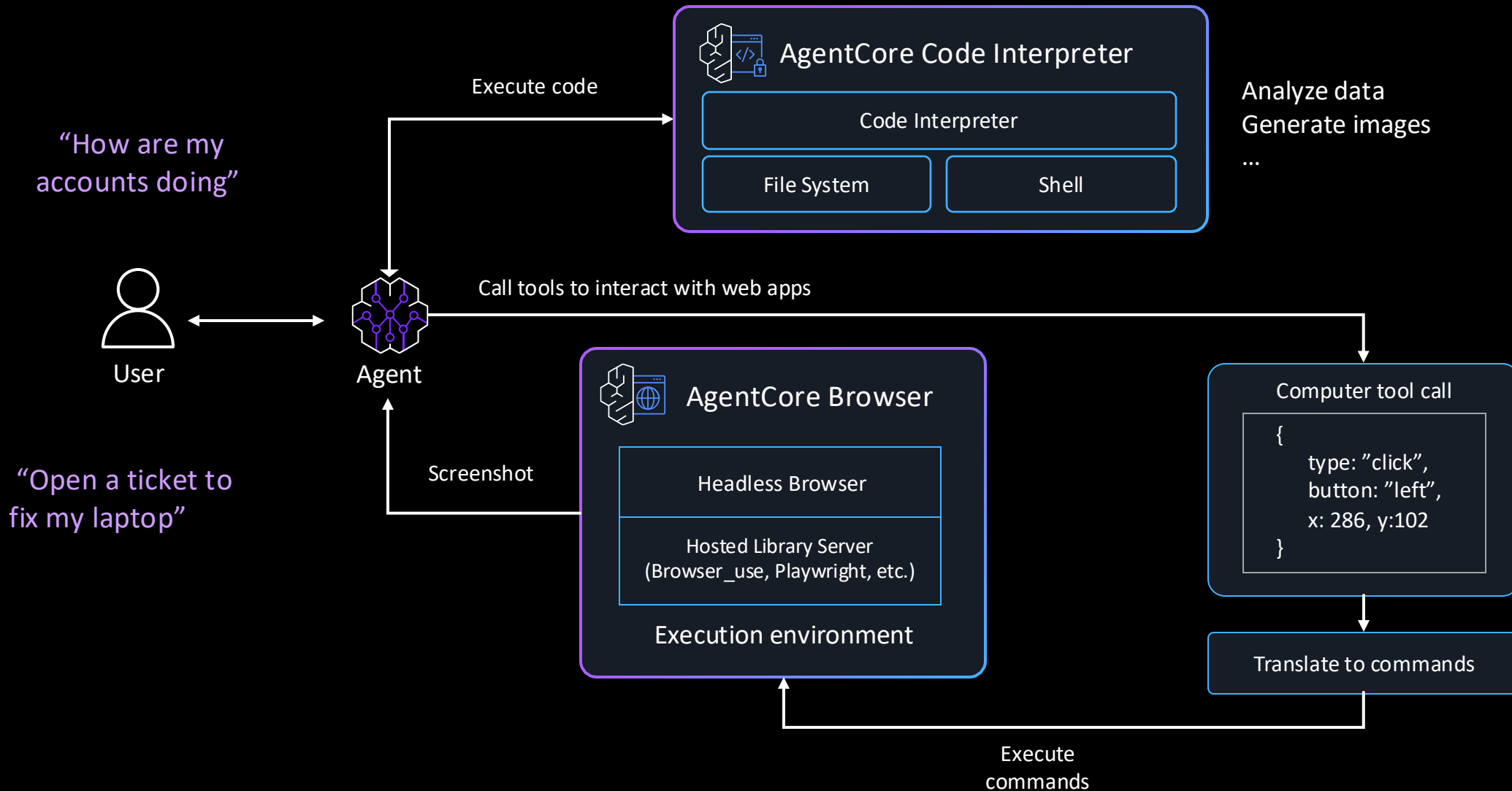


Expose your Knowledge Base as a tool any agent framework can discover and use

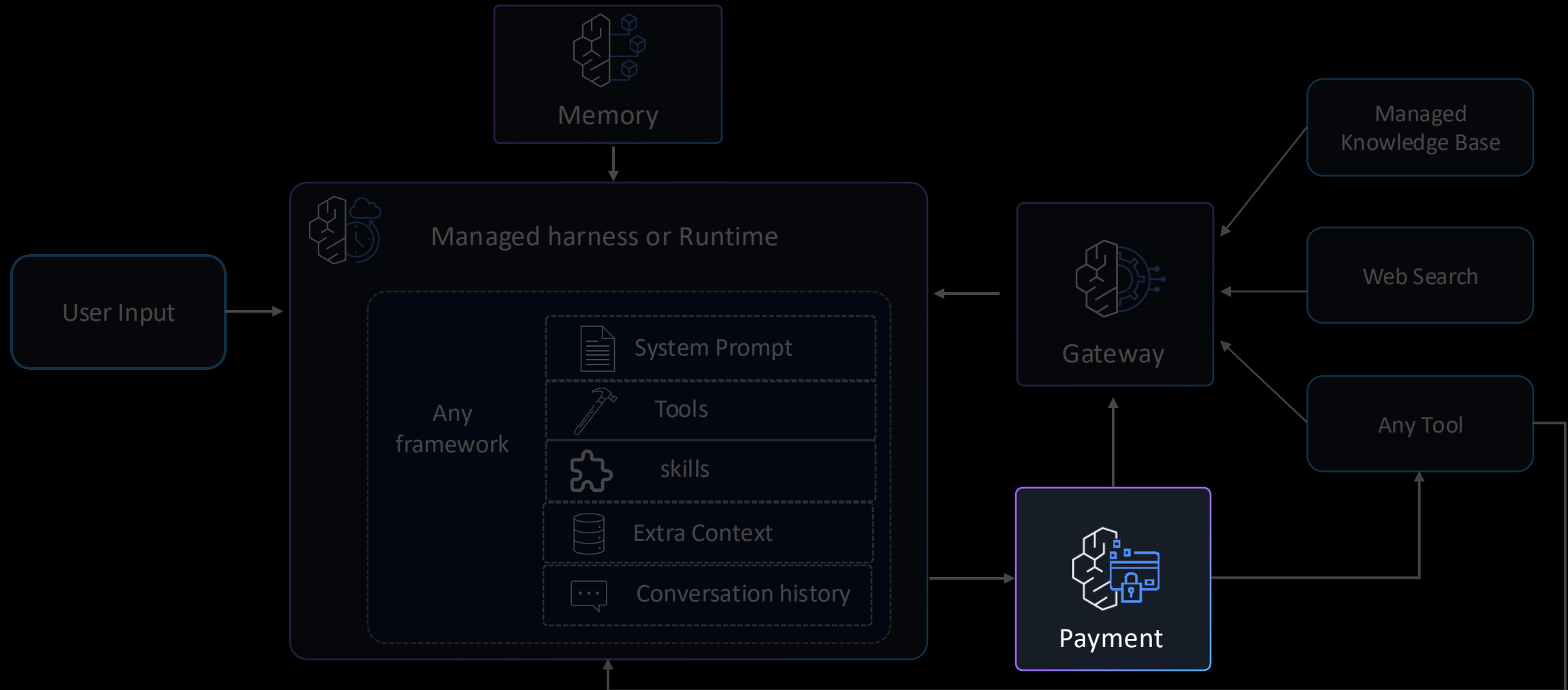
Invoking fully managed tools



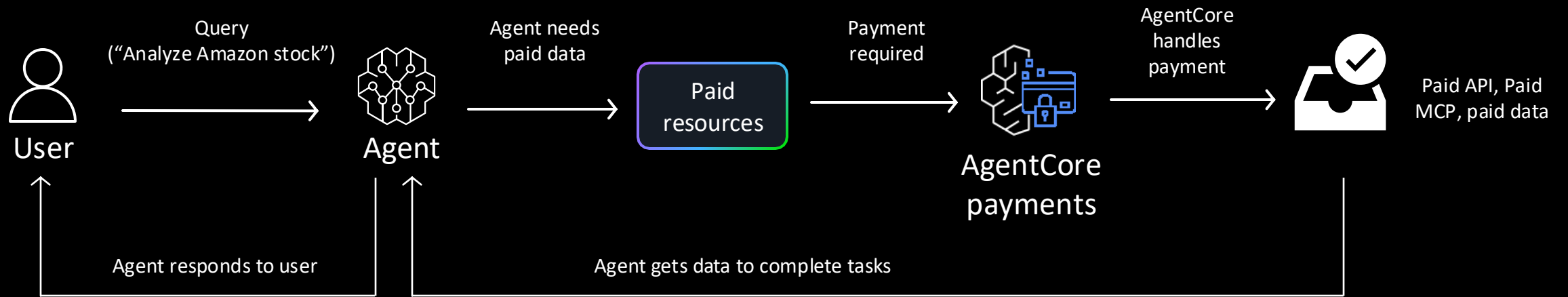
AgentCore Code Interpreter, and Browser



Including paid context to your agent



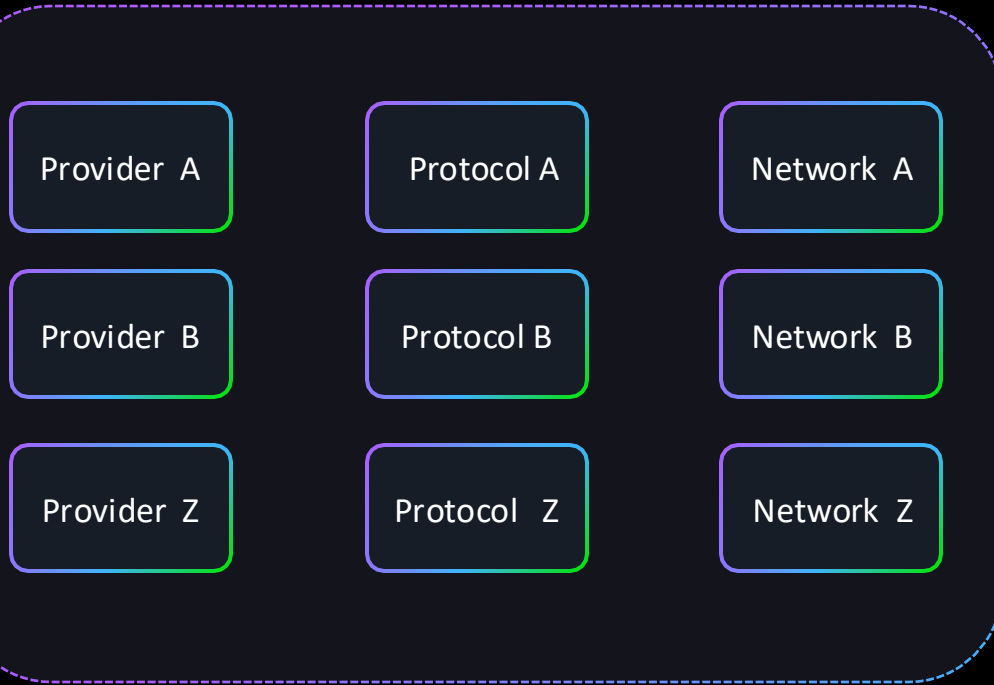
Agents that can reason and act, **now also transact**





AgentCore payments

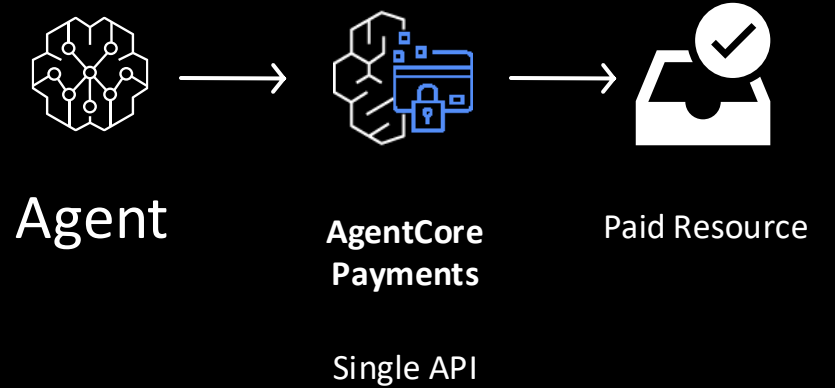
Without



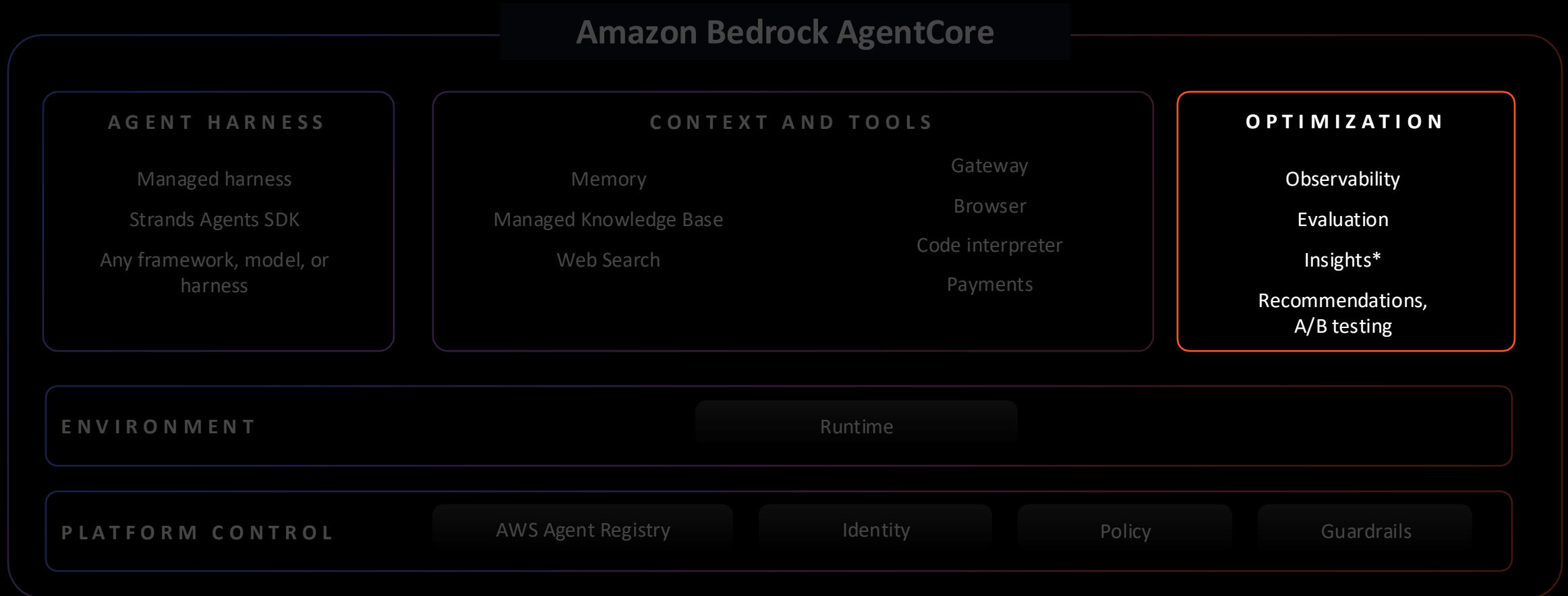
Agent Developer has to build and maintain Providers X protocols X Networks plumbing

Paid Resource

With



How do you make your agents better?



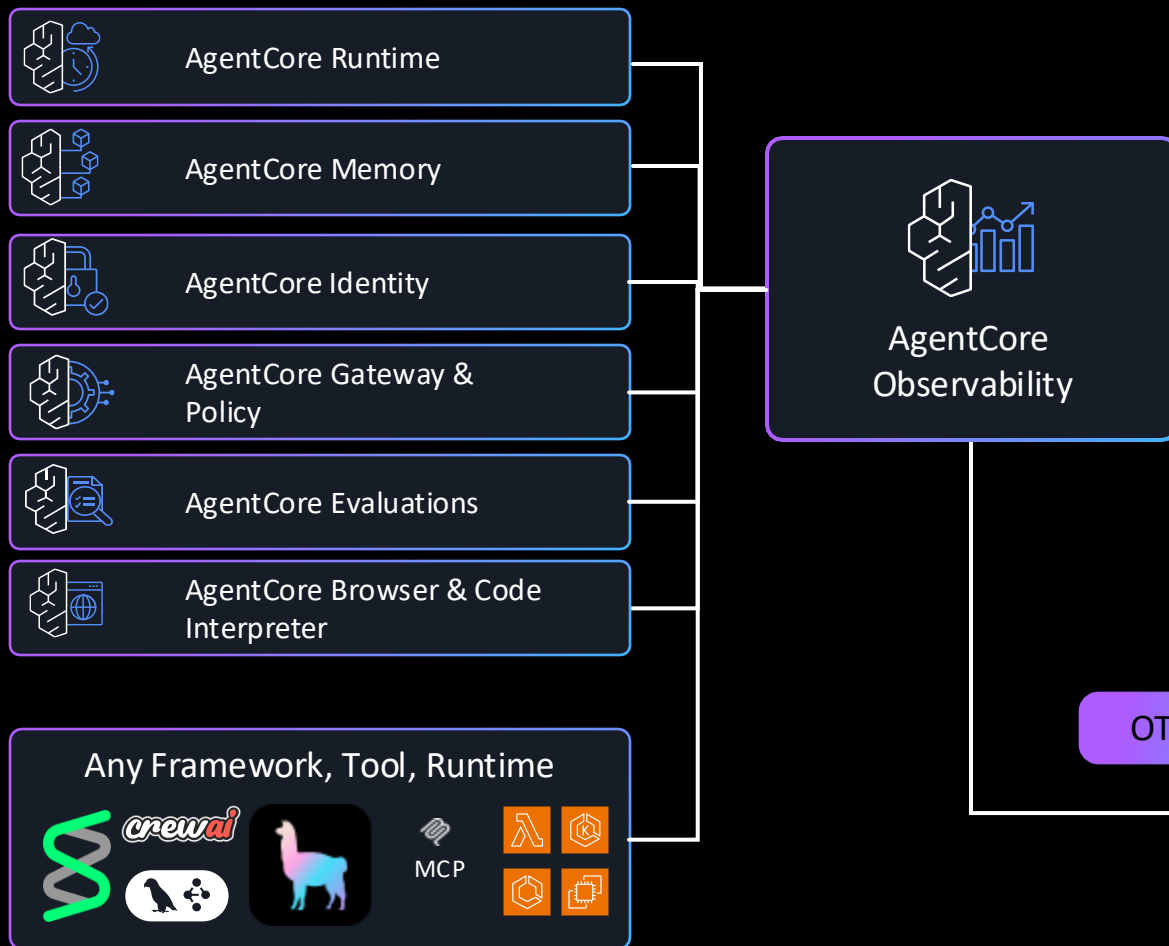
COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features

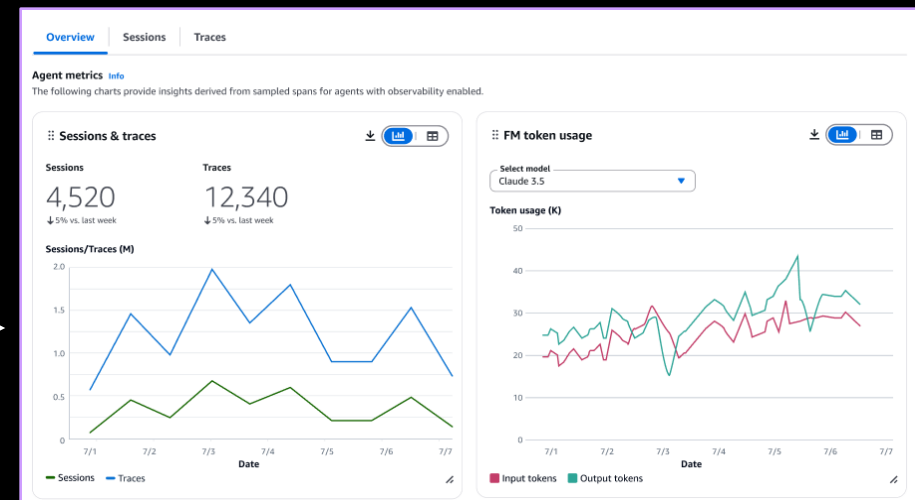




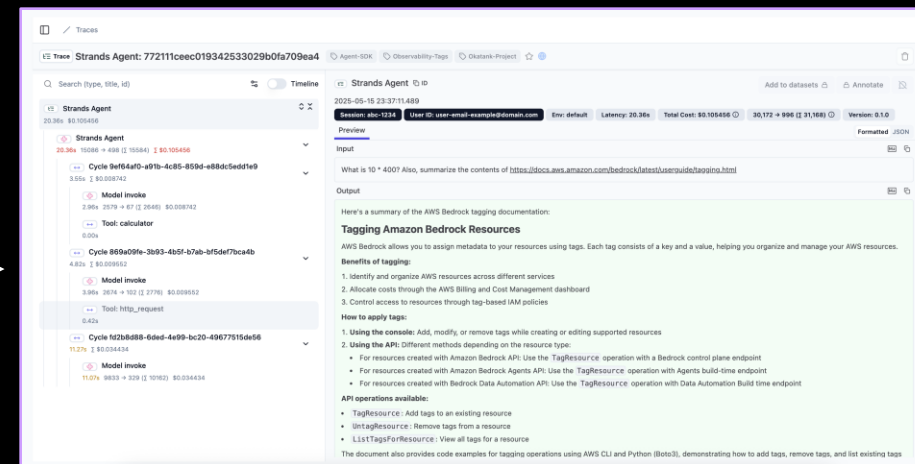
AgentCore Observability



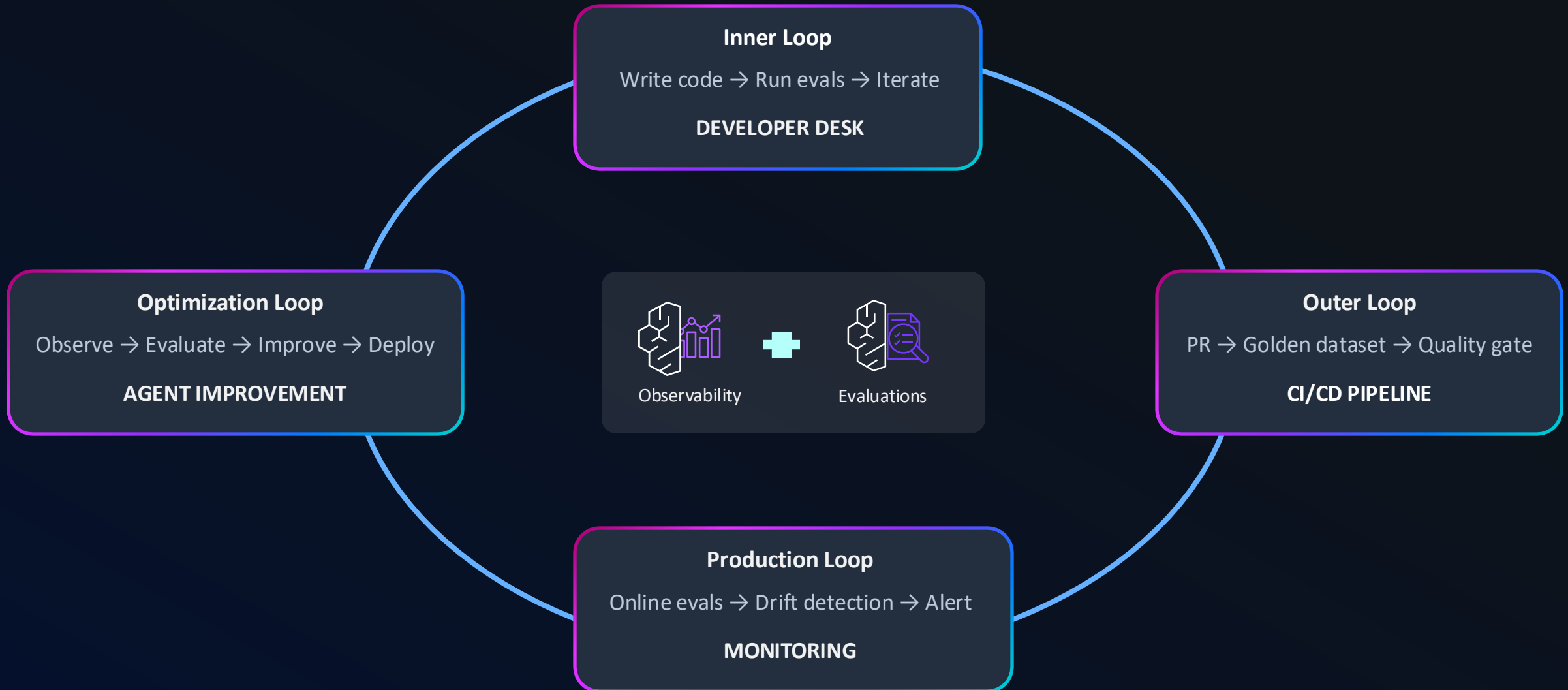
AgentCore Observability dashboards



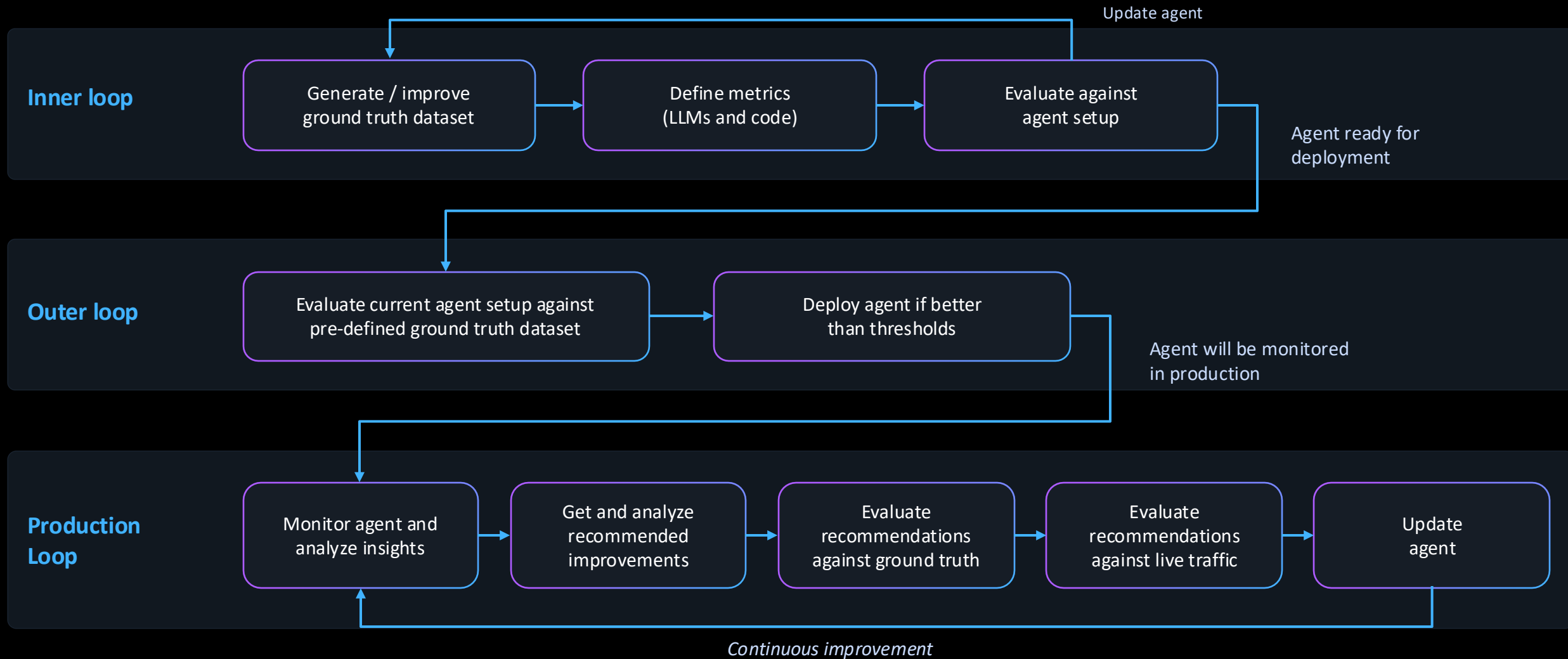
Third-party observability dashboards



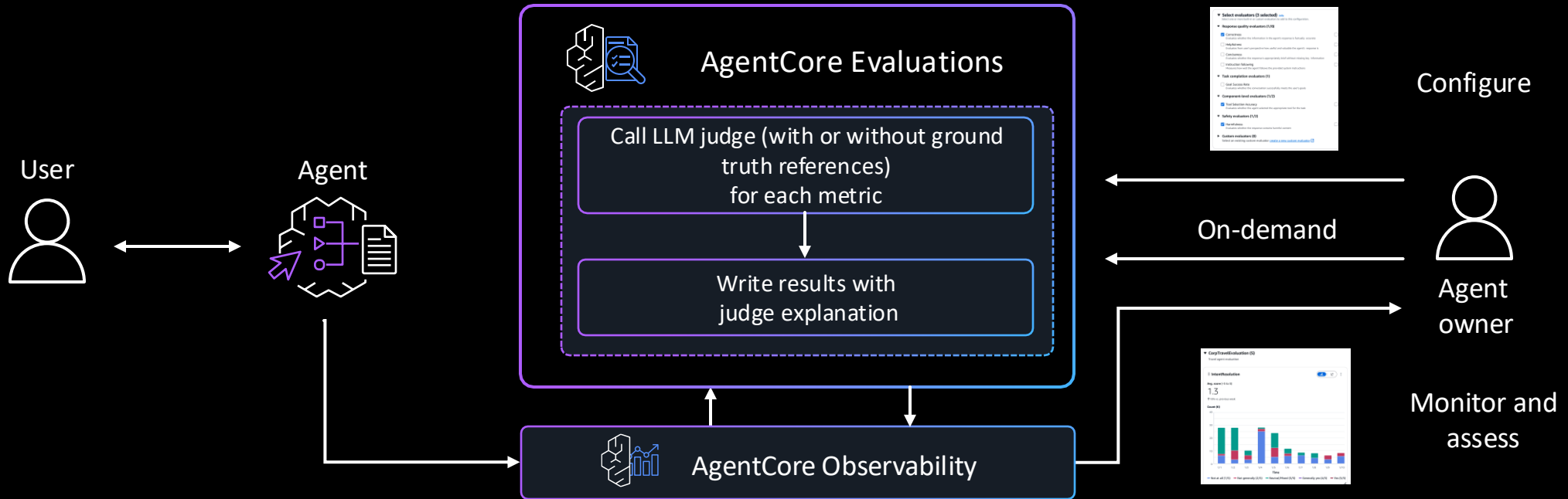
End-to-end agent evaluation and optimization loops



End-to-end agent evaluation and optimization loops



AgentCore Evaluations



Continuously inspect quality

Real-world behavior
evaluated in production

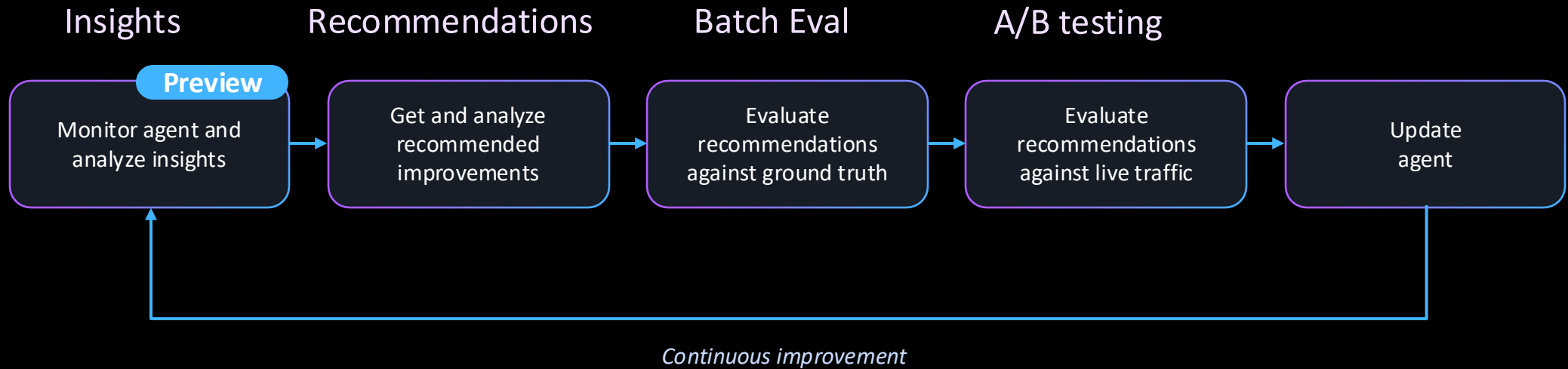
LLM-as-Judge and Code Based evaluations

Comes with 13 evaluators **with or without ground truth** references,
and define your own as LLM-as-Judge or via code

On-demand evaluation

Validate before production,
ensure smooth upgrades

AgentCore optimization loop



Recommendations in AgentCore optimization

AI-generated improvements based on real agent traces and eval outputs

System Prompt Recommendation

- Analyzes agent traces for failure patterns
- Generates optimized system prompt
- Adds targeted behavioral instructions
- Improves performance on target evaluator

Tool Description Recommendation

- Analyzes tool selection and usage patterns
- Generates optimized tool descriptions
- Improves tool selection accuracy
- Better parameter handling

Batch evaluation in AgentCore evaluations

1 Invoke

- Run all scenarios concurrently
- Each scenario gets a unique session ID
- Framework-agnostic callable

2 Wait

- Configurable ingestion delay
- Allows CloudWatch to ingest OTEL spans
- Paid once for the full batch

3 Submit

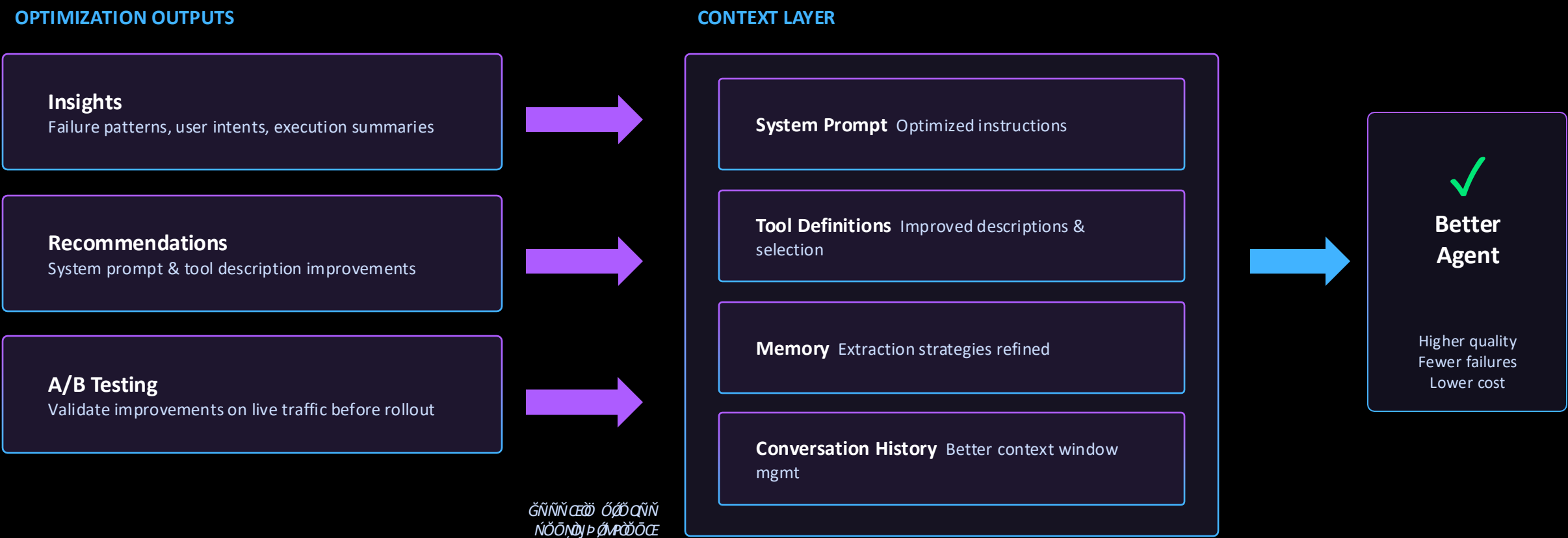
- Calls StartBatchEvaluation API
- Passes log group, session IDs, evaluators
- Service handles span download + eval

4 Poll

- Polls until evaluation completes
- Returns aggregate scores per evaluator
- Per-session detail in CW log stream

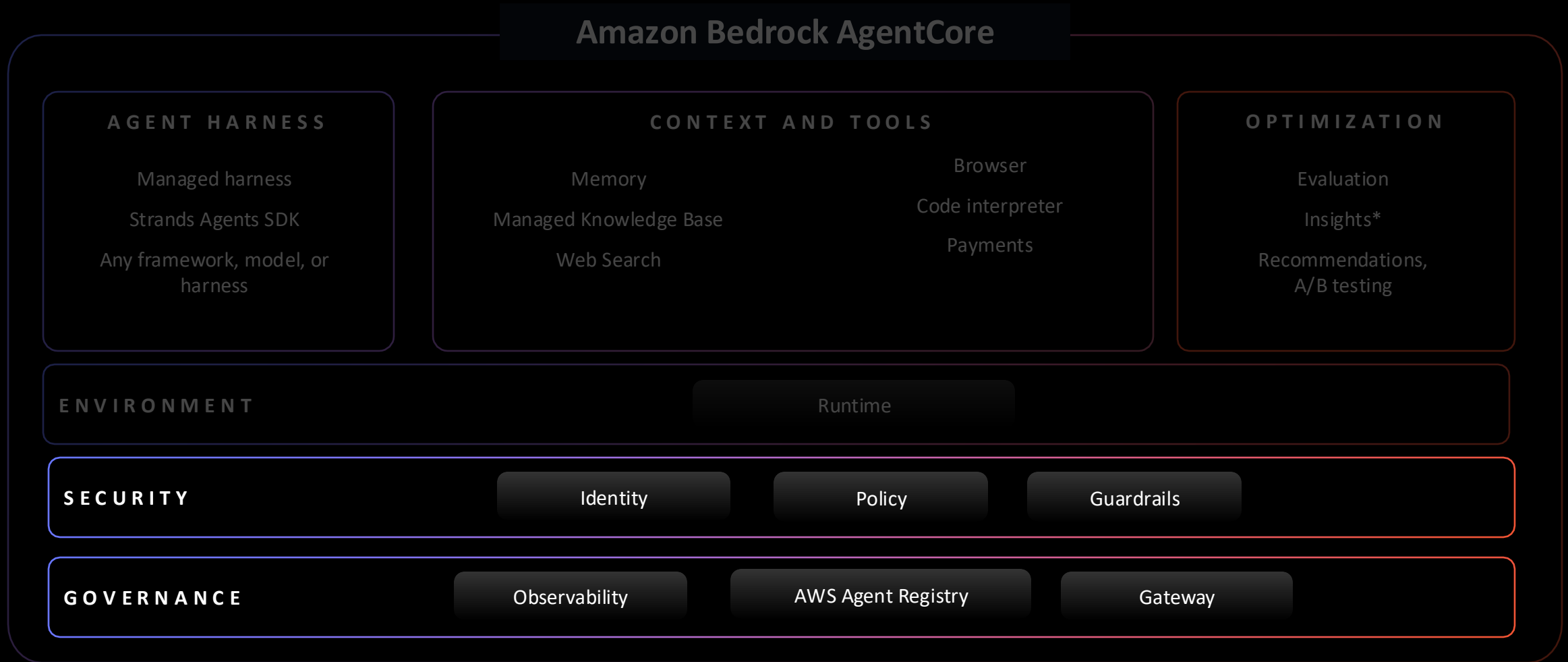
Optimization feeds back into context

The optimization loop directly improves how your agent assembles knowledge



As agents improve autonomously, every interaction needs deterministic control → **Governance & Security**

Securing and governing agent's interactions



COMPOSABLE BY DESIGN. PAY ONLY FOR WHAT YOU USE.

*Preview features



Govern and secure agents across two planes

GOVERNANCE PLANE

Define policy & manage lifecycle

Registry & Catalog

Identity & Access

Observability & Audit

Testing & Evaluation

Data Governance



ENFORCEMENT PLANE

Enforce at runtime, every request

Agent Runtime & Model Catalog

Guardrails

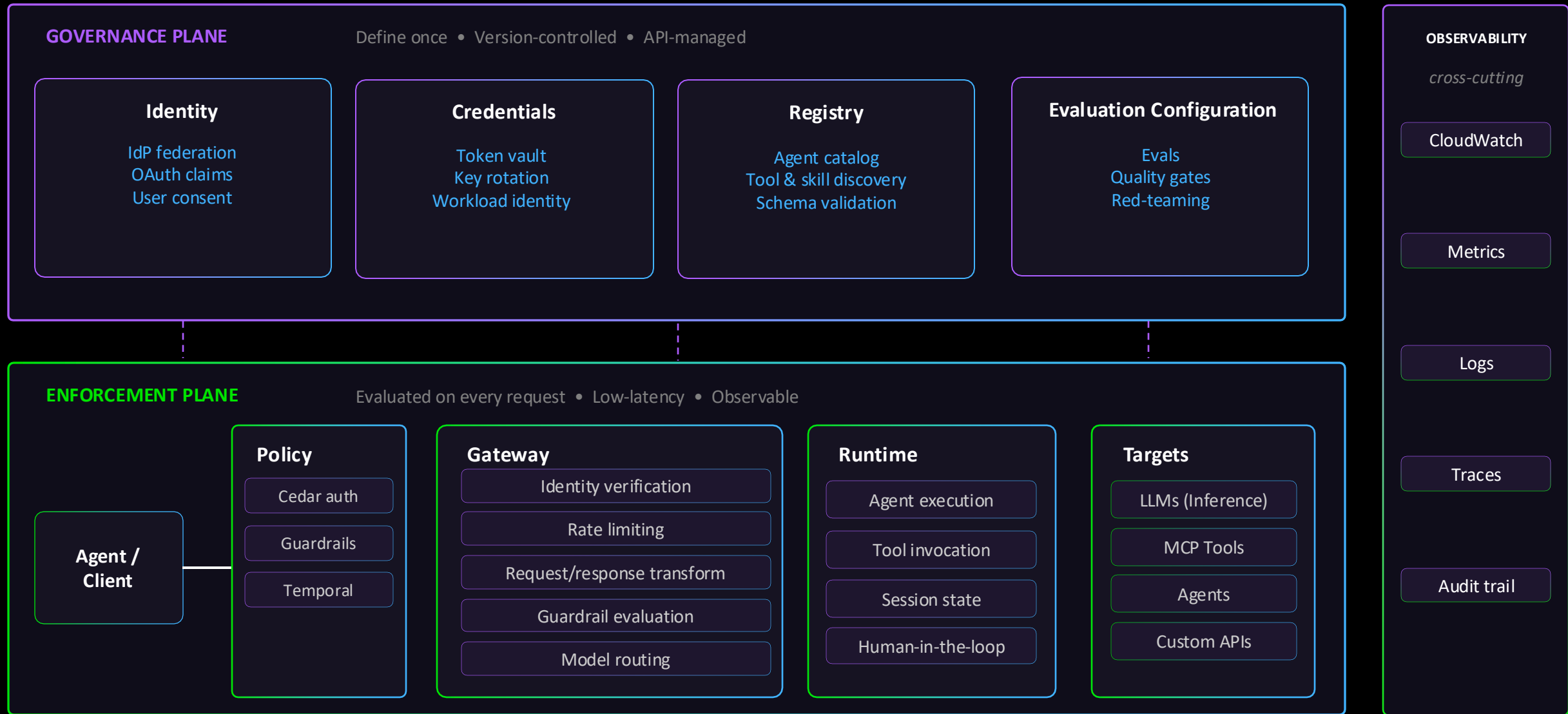
Human-in-the-Loop

Gateway to enforce access

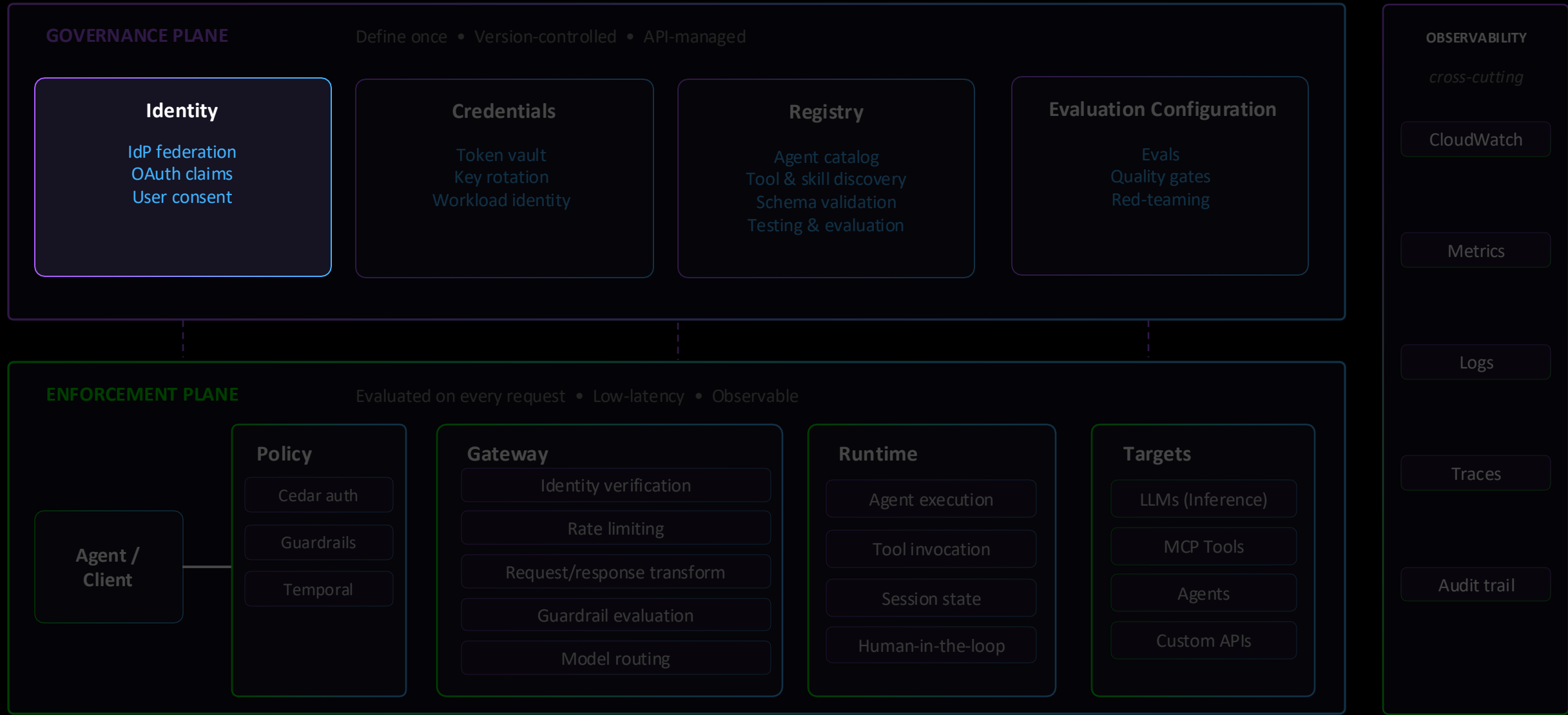
Policy Engine

Governance Plane defines what's allowed • Enforcement Plane applies those decisions in real time

Separate what you configure from what executes on every request

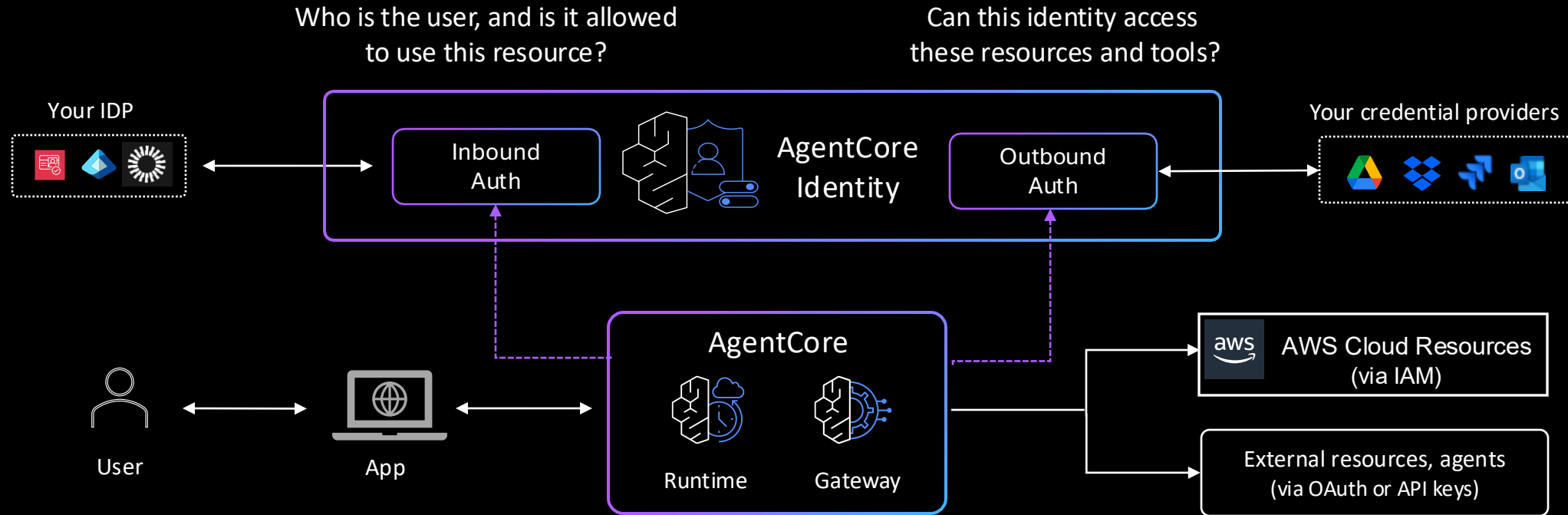


Handling the Identity from your users and from your agents





AgentCore Identity



Any provider

Any identity provider or
credential provider

Secure

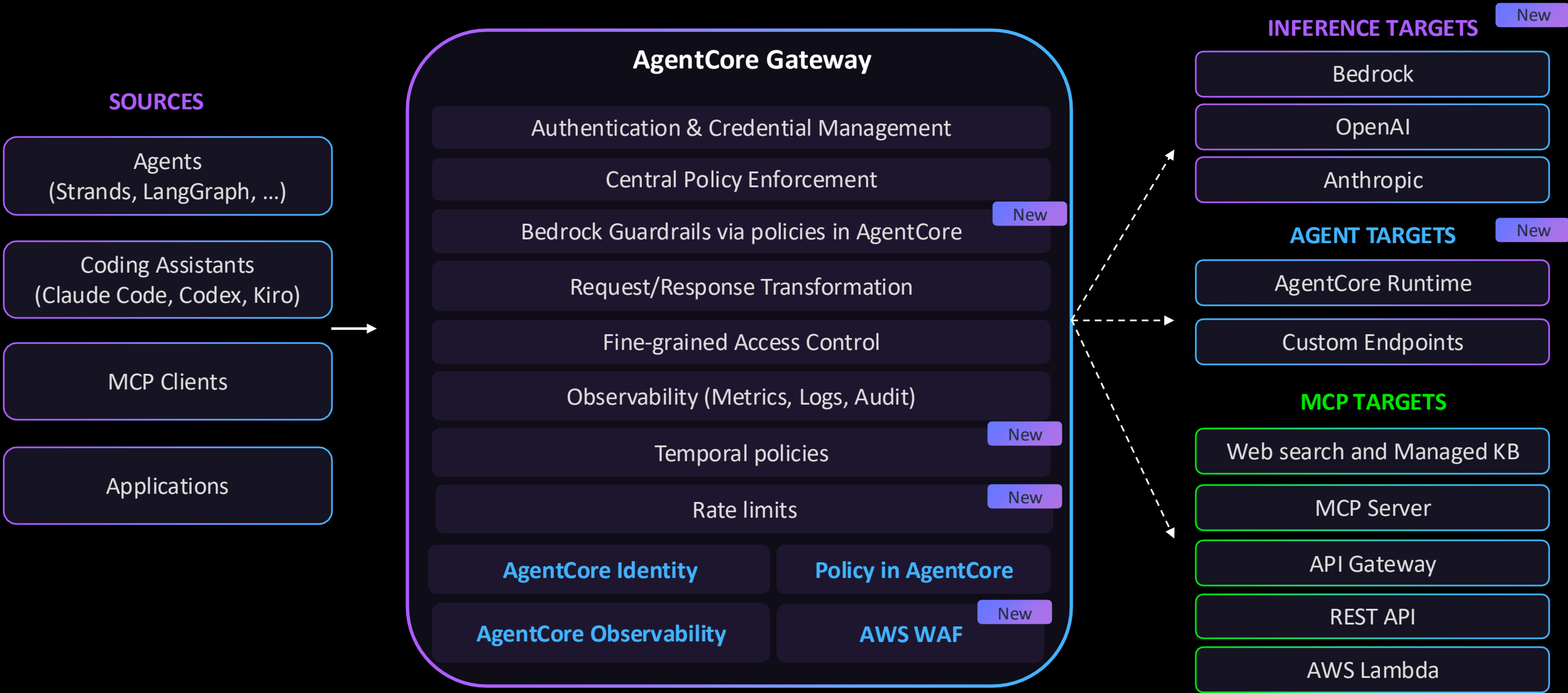
Strict controls and audit
trails

Full coverage

Standard OAuth flows, API
keys, IAM Sigv4

AgentCore Gateway: Serverless AI Gateway

Connect and govern agents, tools, and models through a single secure endpoint

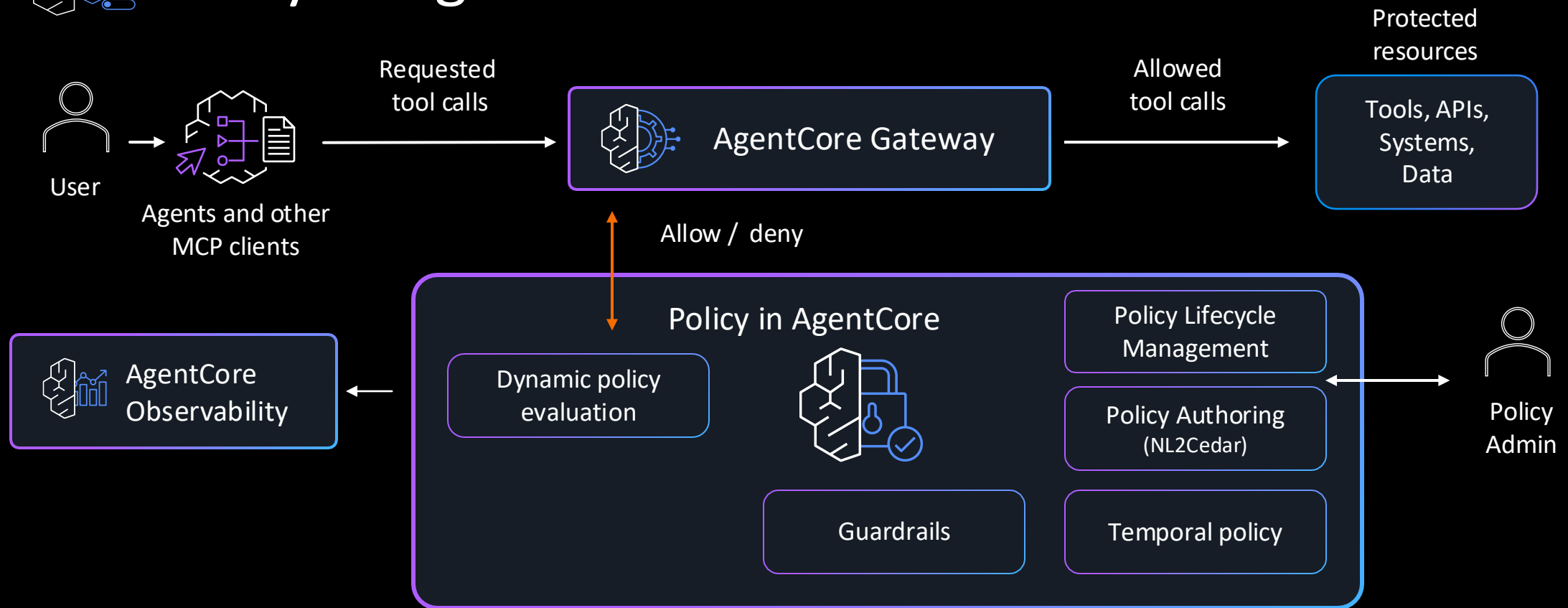


AgentCore Gateway: Guardrails & Policy Enforcement





Policy in AgentCore



Keep agents in bounds

Act autonomously, but stay within boundaries and compliance

Instant and consistent

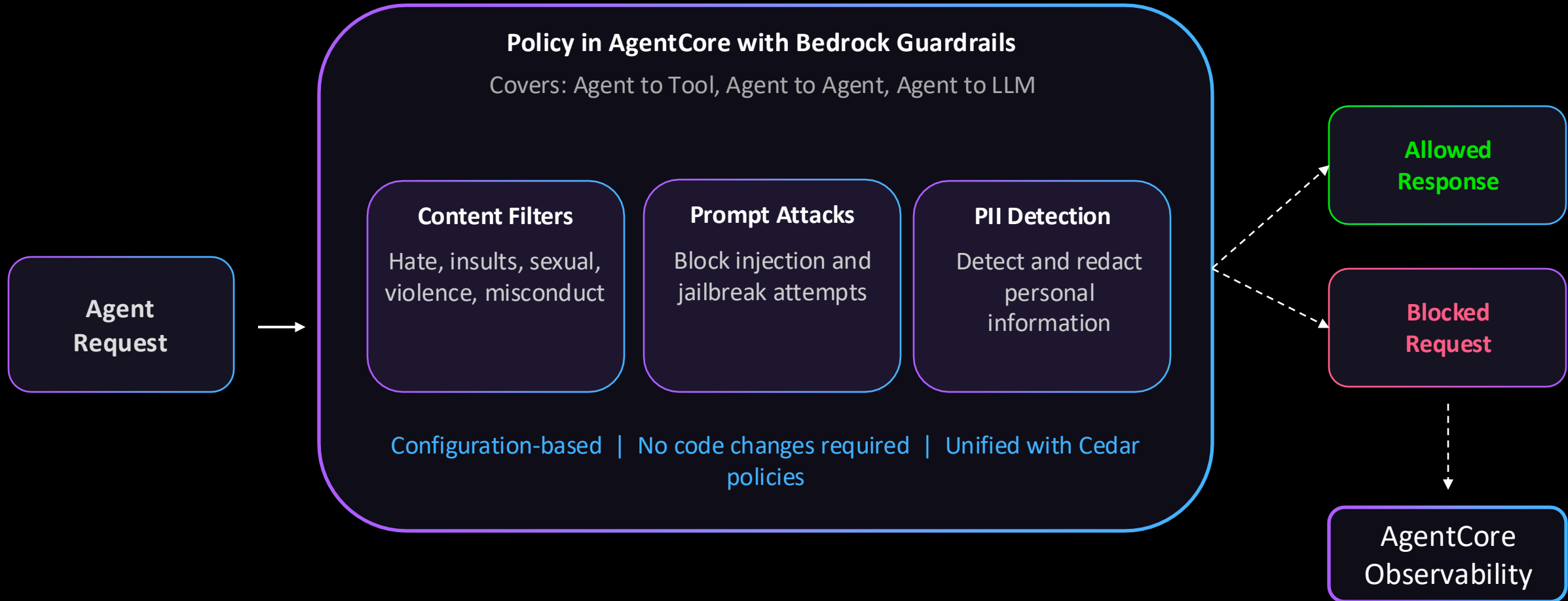
Policies evaluated in milliseconds, keeps agents fast

Verifiably correct

Built on years of automated reasoning, using Cedar

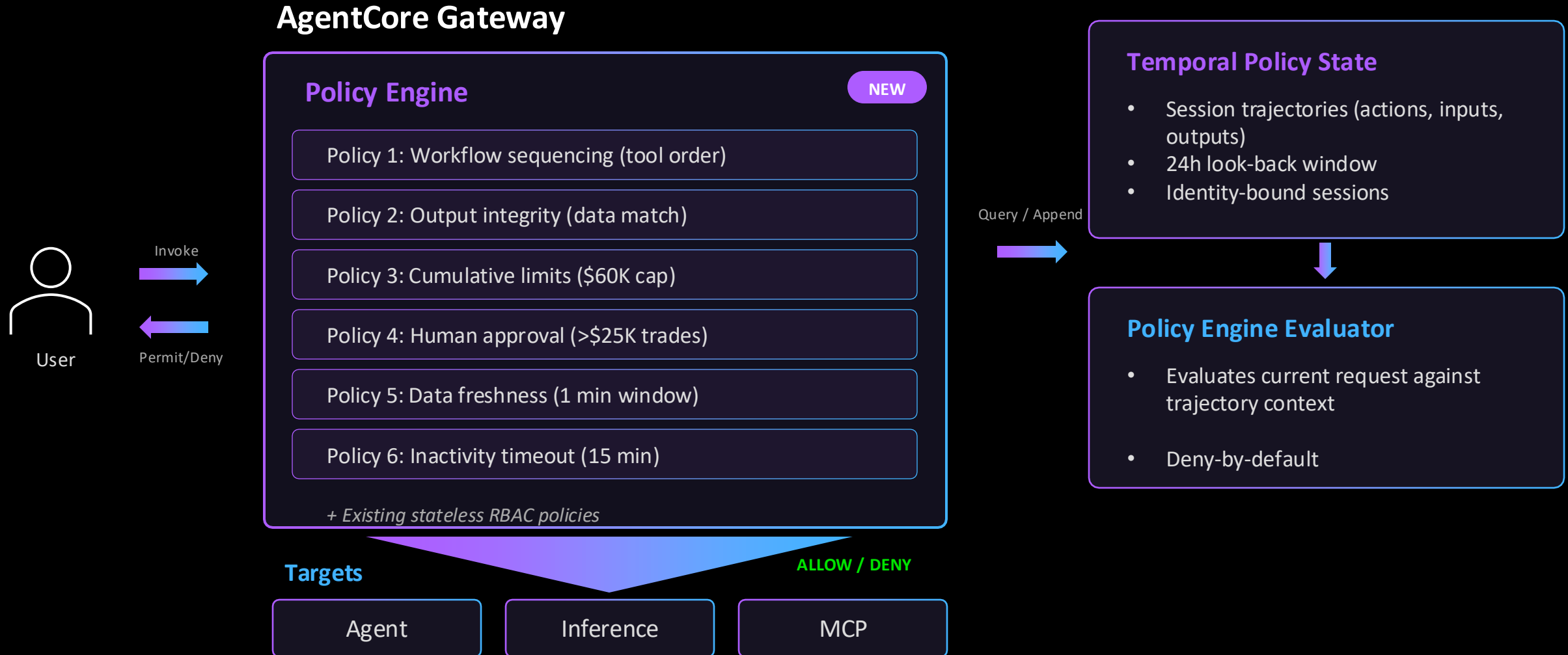
Guardrails in Policy for AgentCore

Attach Amazon Bedrock Guardrails to policy in AgentCore for real-time controls with no code changes



Temporal Policies in AgentCore

Stateful, trajectory-aware authorization that evaluates the agent's session history at the gateway perimeter





Q&A



Thank you.